



**INSTITUTE OF SCIENCE &
TECHNOLOGY FOR
ADVANCED STUDIES & RESEARCH**
(A CONSTITUENT COLLEGE OF THE CVM UNIVERSITY)



Sardar Patel Centre of Science & Technology, P.O. Box No. 13, Vallabh Vidyanagar-388120, Dist. Anand (Gujarat) INDIA
Ph: 02692-234955; E-Mail: principal.istar@cvmu.edu.in; Web: www.istar.edu.in

Master of Computer Application (MCA) Semester – I

Subject Name: Python – Beginner to Professional

Prepared by:
Mr. Yash Panchal
Assistant Professor,
ISTAR, V.V.Nagar, Gujarat
yash.panchal@cvmu.edu.in

Unit- 1

- **Introduction:**
- What is Python?, Paradigms of Programming, Procedural Programming Model, Object
- Programming Model, Third-party Packages
- **Python Basics:**
- Identifiers and Keywords, Data types and assignment, Arithmetic Conversions, Built-in functions and modules
- Console Input / Output: Console Input: split(), Console Output
- String: Accessing String Element, Properties, Built Comparison
- **Decision Control Instruction:** if block, else block, elif block, Logical Operators, Conditional Expression, pass Statement Repetition Control Instruction: for Statement, while Statement, break and continue Statement

What is Python?

- Python is a popular high-level programming language used in various applications
 - Python is an easy language to learn because of its simple syntax
 - Python can be used for simple tasks such as plotting or for more complex tasks like machine learning



Brief History of Python

- Invented in the Netherlands, early 90s by **Guido van Rossum**
- Named after Monty Python
- Open sourced from the beginning
- Considered a scripting language, but is much more
- Scalable, object oriented and functional from the beginning
- Used by Google from the beginning
- Increasingly popular

Python's Benevolent Dictator For Life

- “ Python is an experiment in how much freedom programmers need. Too much freedom and nobody can read another's code; too little and expressive-ness is endangered.”
- Guido van Rossum



What is Python?

Python is a high-level, general-purpose, and very popular programming language. Python programming language (latest Python 3) is being used in web development, Machine Learning applications, along with all cutting-edge technology in Software Industry.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard library which can be used for the following:

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like OpenCV, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia
- Scientific computing
- Text processing and many more..

Introduction to Python

Python is a widely used general-purpose, high level programming language.

It was created by Guido van Rossum in 1991 and further developed by the Python Software Foundation.

It was designed with an emphasis on code readability, and its syntax allows programmers to express their concepts in fewer lines of code.

Python is a programming language that lets you work quickly and integrate systems more efficiently.

There are two major Python versions:

Python 2 and Python 3. Both are quite different.

Reason for increasing popularity

1. Emphasis on **code readability**, **shorter codes**, ease of writing
2. Programmers can express logical concepts in **fewer lines** of code in comparison to languages such as C++ or Java.
3. Python supports **multiple** programming paradigms, like object-oriented, imperative and functional programming or procedural.
4. There exists inbuilt functions for almost all of the frequently used concepts.
5. Philosophy is “Simplicity is the best”.

LANGUAGE FEATURES

- **Interpreted**

- There are no separate compilation and execution steps like C and C++.
- Directly *run* the program from the source code.
- Internally, Python converts the source code into an intermediate form called bytecodes which is then translated into native language of specific computer to run it.
- No need to worry about linking and loading with libraries, etc.

- **Platform Independent**

- Python programs can be developed and executed on multiple operating system platforms.
- Python can be used on Linux, Windows, Mac and many more.

- **Free and Open Source; Redistributable**

LANGUAGE FEATURES

- **High-level Language**

- In Python, no need to take care about low-level details such as managing the memory used by the program.

- **Simple**

- Closer to English language; Easy to Learn
- More emphasis on the solution to the problem rather than the syntax

- **Embeddable**

- Python can be used within C/C++ program to give scripting capabilities for the program's users.

- **Robust:**

- Exception handling features
- Memory management techniques in built

LANGUAGE FEATURES

- **Rich Library Support**

- The Python Standard Library is very vast.
- Known as the “**batteries included**” philosophy of Python ;It can help do various things involving regular expressions, documentation generation, unit testing, threading, databases, web browsers, CGI, email, XML, HTML, WAV files, cryptography, GUI and many more.
- Besides the standard library, there are various other high-quality libraries such as the Python Imaging Library which is an amazingly simple image manipulation library.

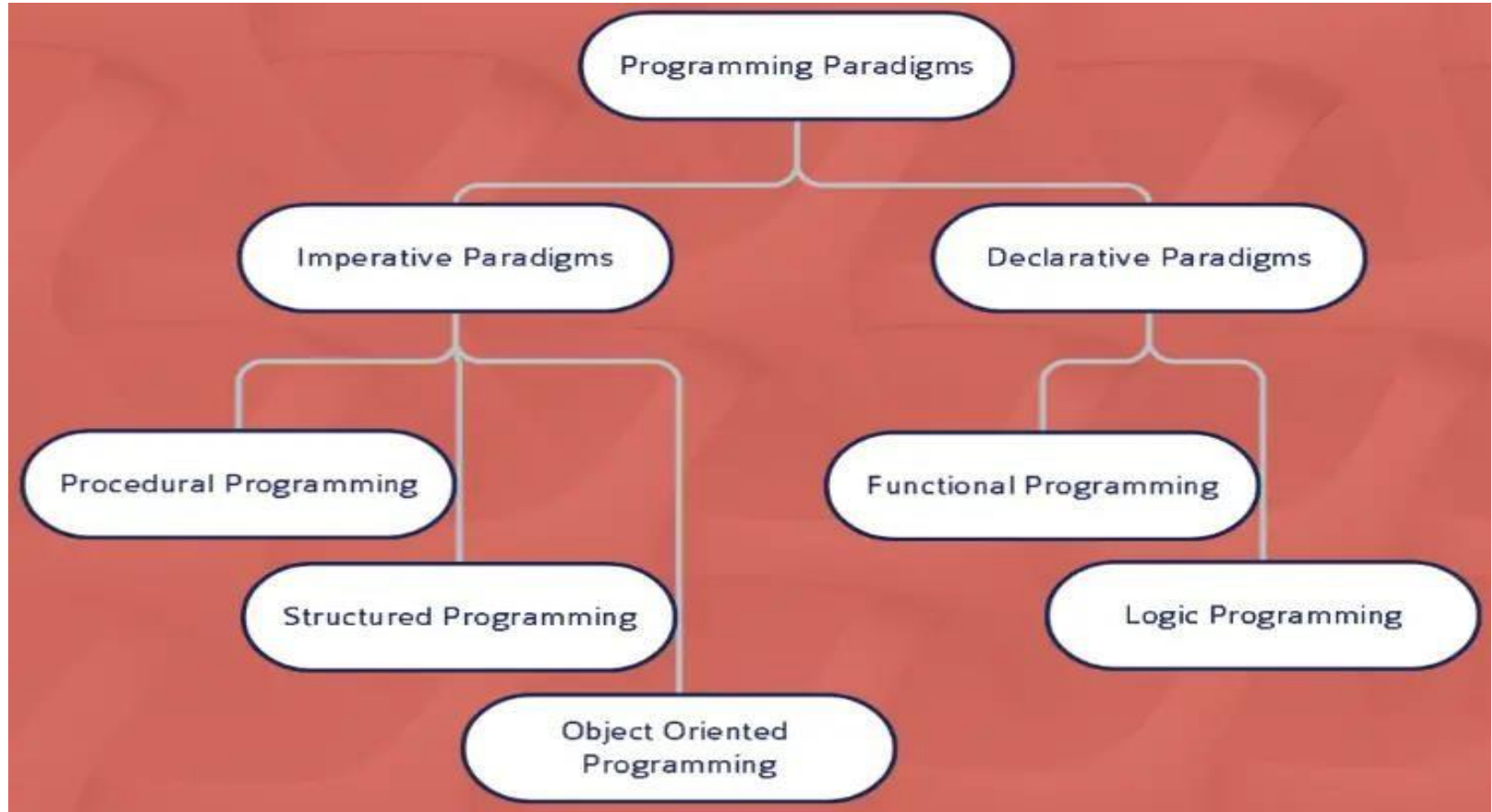
Types of Programming Paradigms

- Every day, we come across different programming languages that we use for specific purposes. A program can be built in multiple ways to provide solutions to a problem.

What Is a Programming Paradigm?

- A programming paradigm is the process of writing code in an organized manner specified by the language. Each language is used as a tool for solving a problem and has its own unique style that uses a programming paradigm.
- A program is like a machine. It has many components and parts to work efficiently. This seems great at the start, but this complexity can become your own enemy while translating to real-world problems to provide simple and elegant solutions.
- There are different methods proposed by computer scientists that we can follow to create efficient code for our problem statement.

Types of Programming Paradigms



Types of Programming Paradigms

- In the modern programming world, most languages are multi-paradigm languages.
- A programming language can use the aspects of different paradigms so that we get maximum advantages from all paradigms.
- For example, **Python is a mixture of procedural, object-oriented, and functional programming paradigms.**
- The language is usually coded using the imperative style but can use the declarative style if required.

Imperative Programming

- When you go shopping, you usually write down the name of the things you need to buy before reaching the mall. Similarly, this paradigm consists of a list of programming statements.
- They are first executed, and then the results are stored in a variable. It is more of a line-by-line instruction given to the computer.
- Let us understand imperative programming using a **C++ program that gives us factorial of a number.**
- In this example, we give a line-by-line task to perform by the computer with proper instructions.
- This type of paradigm takes the order of the steps into consideration because different arrangements can produce different results.

```
#include<iostream>
using namespace std;
int main(){
    int fact=1;
    fact*=1;
    fact*=2;
    fact*=3;
    fact*=4;
    fact*=5;
    fact*=6;
    fact*=7;
    fact*=8;
    cout<<"Factorial of 8 is : "<<fact;
    return 0;
}
```

```
❖ make -s
❖ ./main
Factorial of 8 is : 40320❖
```

Procedural Programming

- This programming paradigm follows imperative programming with procedural calls. These calls direct the system to perform the required tasks.
- Each procedure or so-called function can have multiple commands to be executed. The function, once defined, can be called as many times as needed to perform the same operation.
- Procedural programming is considered the best for people starting to learn to code. Beginners can start with procedural programming as it is simple, efficient, requires less memory, and is easier to keep track of control flow.
- Some of the languages that use procedural programming paradigms are:
 - ☐ C
 - ☐ C++
 - ☐ Java
 - ☐ Python
 - ☐ Pascal

Procedural Programming

- Example program in C++:

```
#include<iostream>
using namespace std;
int main(){
    int fact=1;
    for(int i=1;i<=8;i++)
        fact=fact*i;
    cout<<"Factorial of 8 is : "<<fact;
    return 0;
}
```

```
❯ make -s
❯ ./main
Factorial of 8 is : 40320❯
❯
```

Structured Programming

- It is an imperative programming paradigm where we use nested loops, conditional statements, and subroutines to define the control flow of a program.
- This paradigm has a top-down implementation which promotes code readability and reusability. In commercial applications, this helps to reduce development time and increase code quality.
- The disadvantage of structured programming is that it has reduced execution efficiency compared to the other paradigms. Structured programming languages are at risk of “over-structuring” and loss of efficiency.
- Some of the languages that use Structured programming paradigms are:
 - ❑ Algol 60
 - ❑ PL/I
 - ❑ Pascal
 - ❑ Ada 83
 - ❑ Modula

Object-Oriented Programming

- This is the most widely used and most popular programming paradigm.
- Class, Abstraction, Encapsulation, Inheritance, and Polymorphism form the backbone of Object-Oriented Programming (OOP).
- In this paradigm, every entity in the program is considered an object. Objects can have data associated with them called attributes and actions they can perform using methods.
- Some of the languages that use object-oriented programming paradigms are:

- ☐ Python
- ☐ Ruby
- ☐ Java
- ☐ C++
- ☐ Smalltalk

OOP feature	Description
Abstraction	It helps in hiding unnecessary attributes while showing the ones required.
Inheritance	It helps in establishing hierarchical relationships promoting code reusability.
Polymorphism	It allows different objects to respond in different ways to the same input.
Encapsulation	It is the binding of data into a single unit.

Declarative Programming

- Declarative programming is a programming approach that expresses a computation's logic without discussing its control flow. The programmer must specify what the program must accomplish but need not specify how it must be implemented.
- In other words, rather than educating on “how to reach a goal”, declarative programming concentrates on “what needs to be accomplished first”.
- The primary distinction is that imperative commands tell you how to do, while declarative commands tell you what to do. The declarative programming paradigm is further divided into logic programming and functional programming.

Logic Programming

- The logic programming paradigm uses logic-based statements to convey facts and rules. Logical inferences are made of instructions or logic statements to do computation. Atomic statements are built using these predicate statements.
- While displaying relevant data, logic languages frequently rely on queries. Some of the specific uses of logic programming are natural language processing, database management, predictive analysis, etc.
- Some of the languages that use logic programming paradigms are:
 - ☐ Prolog
 - ☐ GHC
 - ☐ Polka
 - ☐ Vulcan
 - ☐ Mercury

Functional Programming

- Functional programming is built upon mathematical functions. It solves problems using these mathematical functions as program components. This programming is used while working with concurrency and parallelism.
- The paradigm algorithm consists of two components: logic and control. The logic includes facts and rules, and the control contains the order of those rules. The logic specifies what needs to be solved, whereas the control specifies how it should be done.
- The advantage of using functional programming is that these programs can run efficiently in multi-threaded and multi-core environments. The programmer only needs to specify the logic and the system automatically finds the best solution.

Functional Programming

- You can implement things in less number of lines using functional programming. It is also easy to unit test and debug using functional programming.
- The disadvantage of functional programming is that there is no for loop and while loops. Instead, we have recursions to iterate through our code.
- Example: **Javascript code that prints “Hello World” 10 times using functional programming.**
- Some of the languages that use Functional programming paradigms are:
 - ❑ Haskell
 - ❑ JavaScript
 - ❑ Scala
 - ❑ Erlang
 - ❑ Lisp

```
Array(10).fill("Hello World")
  .map((msg) => {
    console.log(msg);
    return msg;
  });
```

[illegible]

Python Language Overview:

- Python Readability - **Variables**:

Python:

```
1 is_valid = True
```

PHP:

```
1 $isValid = true;
```

JavaScript:

```
1 let isValid = true;
```

Java / C# / C++ (Strongly Typed Languages):

```
1 Boolean isValid = true
```

Python Language Overview:

- Python Readability - **Control Structures:**

Python:

```
1 price = 200
2 if price == 15:
3     """ Do something"""
4 elif price == 30:
5     """Do something else"""
6 else:
7     """ Do something else"""
```

PHP:

```
1 $price = 200;
2 if ($price == 15){
3     //Here
4 } elseif ($price == 30){
5     //Here
6 } else {
7     // Here
8 }
```

Python Language Overview:

- Python Readability - **Ternary:**

Python:

```
1 price = 200 if True else 0
2 print (price)
3 # 200
```

PHP:

```
1 <?php
2 $price = (true) ? 200 : 0;
3 print $price;
4 # 200
```

Ruby:

```
1 price = if true then 200 else 0 end
2 # 200
```

JavaScript:

```
1 let price = true ? 200 : 0;
2 console.log(price);
3 # 200
```

Python Language Overview:

- A simple program written in C++, C, Java and Python. All program prints **"Hello world"**.

Python Program:

```
1 print ( "Hello World")
```

Java Program:

```
1 public class Hello
2 {
3     public static void main(String argv[])
4     {
5         System.out.println("Hello, World!");
6     }
7 }
```

C++ Program:

```
1 #include <iostream>
2 int main()
3 {
4     std::cout << "Hello World" << std::endl;
5     return 0;
6 }
```

C Program:

```
1 #include <stdio.h>
2 int main(int argc, char ** argv)
3 {
4     printf("Hello, World!\n");
5 }
```

<http://docs.python.org/>

Windows: There are many interpreters available freely to run Python scripts like IDLE (Integrated Development Environment) that comes bundled with the Python software downloaded from <http://python.org/>.

The screenshot shows the official Python 3.13.6 documentation page. The browser address bar displays 'docs.python.org/3/'. The page header includes a language dropdown set to 'English', a version dropdown set to '3.13.6', and a '3.13.6 Documentation »' link. There are also links for 'Theme', 'Auto', 'Quick search', 'Go', 'modules', and 'index'. The left sidebar contains a 'Download' section, a 'Docs by version' list (ranging from Python 3.15 down to 2.6), and 'Other resources' like the PEP Index and Beginner's Guide. The main content area is titled 'Python 3.13.6 documentation' and includes a welcome message. Below this, a 'Documentation sections:' header introduces a grid of links: 'What's new in Python 3.13?' (with a note about documents since Python 2.0), 'Installing Python modules' (with a note about third-party modules and PyPI.org), 'Tutorial' (with a note about a tour of Python's syntax and features), 'Distributing Python modules' (with a note about publishing modules for use by other people), 'Library reference' (with a note about standard library and builtins), 'Extending and embedding' (with a note about C/C++ programmers), 'Language reference' (with a note about syntax and language elements), 'Python's C API' (with a note about C API reference), 'Python setup and usage' (with a note about how to install, configure, and use Python), 'FAQs' (with a note about frequently asked questions with answers), 'Python HOWTOs' (with a note about in-depth topic manuals), and 'Deprecations' (with a note about deprecated functionality).

docs.python.org/3/

Python » English 3.13.6 3.13.6 Documentation » Theme Auto Quick search Go modules index

Download
Download these documents

Docs by version

- Python 3.15 (in development)
- Python 3.14 (pre-release)
- Python 3.13 (stable)
- Python 3.12 (security-fixes)
- Python 3.11 (security-fixes)
- Python 3.10 (security-fixes)
- Python 3.9 (security-fixes)
- Python 3.8 (EOL)
- Python 3.7 (EOL)
- Python 3.6 (EOL)
- Python 3.5 (EOL)
- Python 3.4 (EOL)
- Python 3.3 (EOL)
- Python 3.2 (EOL)
- Python 3.1 (EOL)
- Python 3.0 (EOL)
- Python 2.7 (EOL)
- Python 2.6 (EOL)
- All versions

Other resources

- PEP Index
- Beginner's Guide
- Book List
- Audio/Visual Talks

Python 3.13.6 documentation

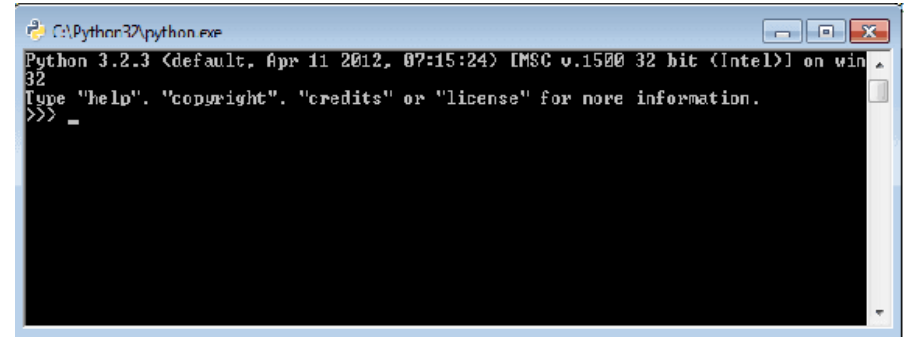
Welcome! This is the official documentation for Python 3.13.6.

Documentation sections:

- [What's new in Python 3.13?](#)
Or all "What's new" documents since Python 2.0
- [Installing Python modules](#)
Third-party modules and PyPI.org
- [Tutorial](#)
Start here: a tour of Python's syntax and features
- [Distributing Python modules](#)
Publishing modules for use by other people
- [Library reference](#)
Standard library and builtins
- [Extending and embedding](#)
For C/C++ programmers
- [Language reference](#)
Syntax and language elements
- [Python's C API](#)
C API reference
- [Python setup and usage](#)
How to install, configure, and use Python
- [FAQs](#)
Frequently asked questions (with answers!)
- [Python HOWTOs](#)
In-depth topic manuals
- [Deprecations](#)
Deprecated functionality

You may open **Python command line** from Start Menu and you may run python commands as shown bellow:

There are some users who prefer command line intersection, rather than a GUI interface. To go Python command line, click on Python 3.2 tab then click on Python(command line).

A screenshot of a Windows command prompt window titled "C:\Python32\python.exe". The window shows the Python 3.2.3 shell prompt. The text inside the window is: "Python 3.2.3 <default>, Apr 11 2012, 07:15:24 [MSC v.1500 32 bit (Intel)] on win32", followed by a blank line, then "Type 'help', 'copyright', 'credits' or 'license' for more information.", and finally the prompt ">>> _".

Here is the screen shot of Python command line interface.
To return type **exit()** or **Ctrl+Z** plus Enter key.

Python IDLE

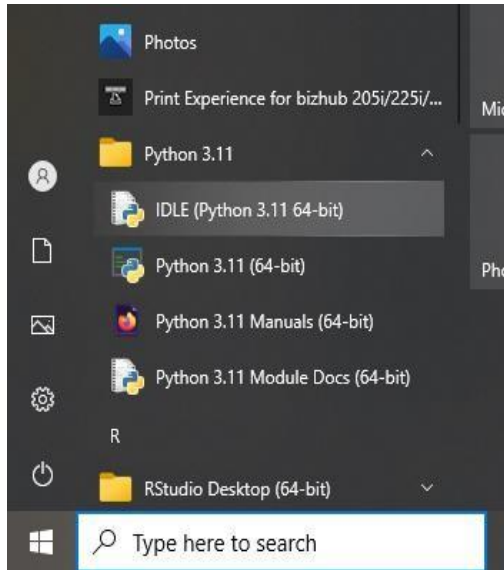
IDLE is an integrated development environment (an application like a word processor which helps developers to write programs) for Python. IDLE is the Python IDE which comes with Python, built with the tkinter GUI toolkit. It has two modes Interactive and Development.

IDLE features:

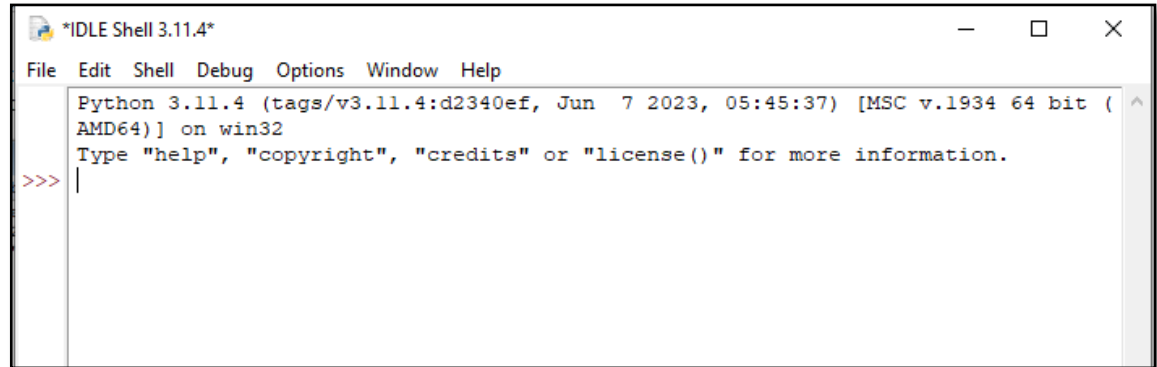
- Cross Platform : Works on Unix and Windows.
- Multi-window text editor with syntax highlighting and smart indent and other.
- Python shell window with syntax highlighting.
- Integrated debugger.
- Coded in Python, using the tkinter GUI toolkit

Python IDLE: Interactive Mode

- Let us assume that we've already installed Python (here we installed Python 3.11 on a standard pc with windows 10 OS). Click on start button and find Python 3.11 tab in installed programs.

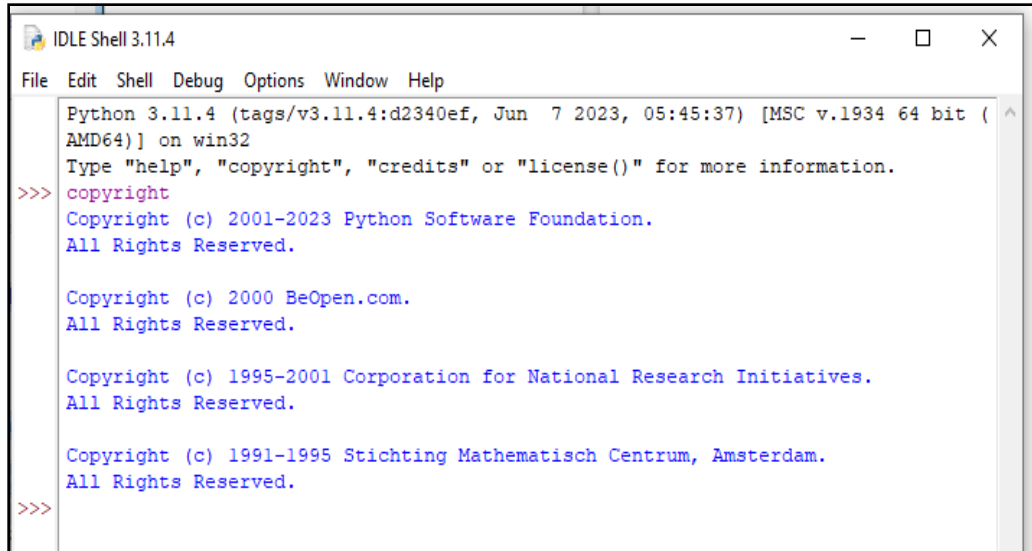


- Now clicking on Python 3.11 tab you can see the Python program development tool named IDLE (Python 3.11 64-bit GUI)
- To start IDLE click on IDLE (Python GUI) icon, you will see a new window opens up and the cursor is waiting beside '>>>' **sign** which is called command prompt.



Python IDLE: Interactive Mode

- This mode is called interactive mode as you can interact with IDLE directly, you type something (single unit in a programming language) and press enter key Python will execute it, but you can not execute your entire program here. At the command prompt type copyright and press enter key Python executes the copyright information.



```
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun 7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> copyright
Copyright (c) 2001-2023 Python Software Foundation.
All Rights Reserved.

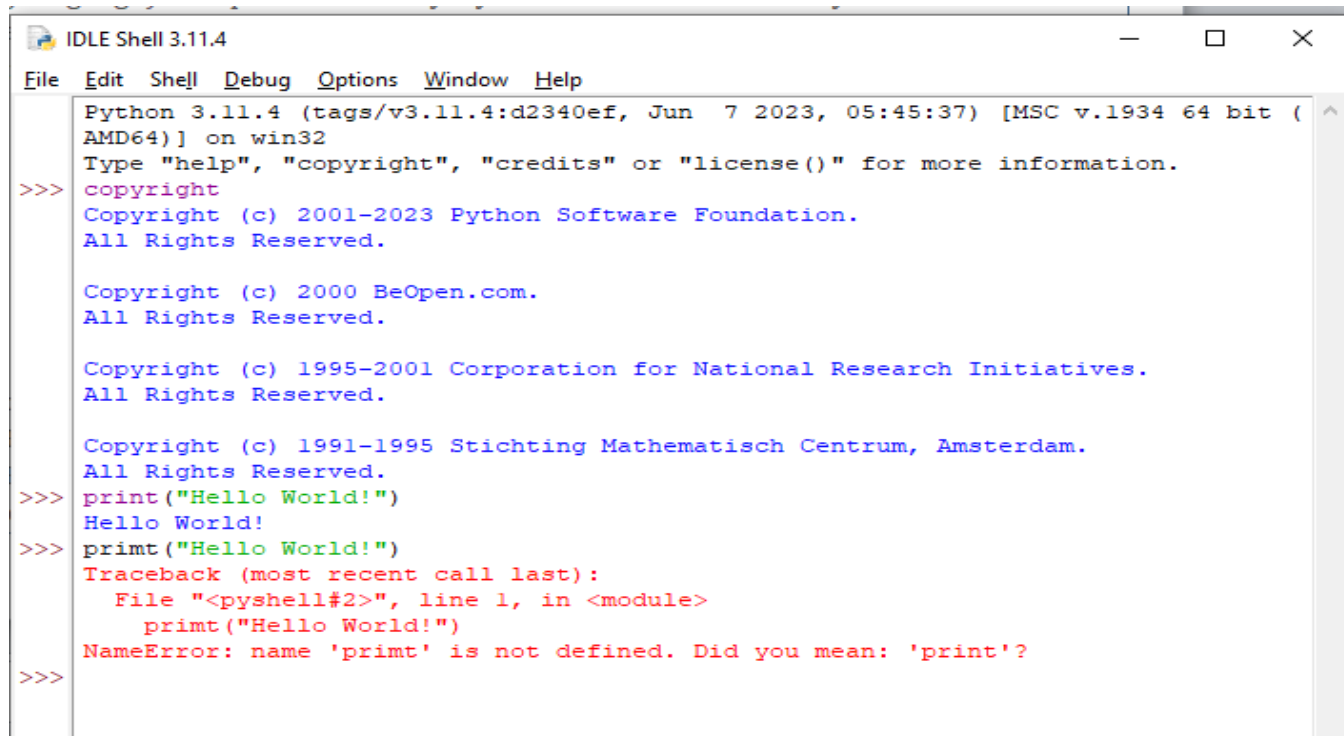
Copyright (c) 2000 BeOpen.com.
All Rights Reserved.

Copyright (c) 1995-2001 Corporation for National Research Initiatives.
All Rights Reserved.

Copyright (c) 1991-1995 Stichting Mathematisch Centrum, Amsterdam.
All Rights Reserved.
>>>
```

Python IDLE: Interactive Mode

- Now Python is ready to read new command. Let's execute the following commands one by one.
- Command -1 : **print**("Hello World")
Command -2 : **primt**("Hello World")
- The first command is correct and but the second one has a syntax error, here is the response from Python.



```
IDLE Shell 3.11.4
File Edit Shell Debug Options Window Help
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun 7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> copyright
Copyright (c) 2001-2023 Python Software Foundation.
All Rights Reserved.

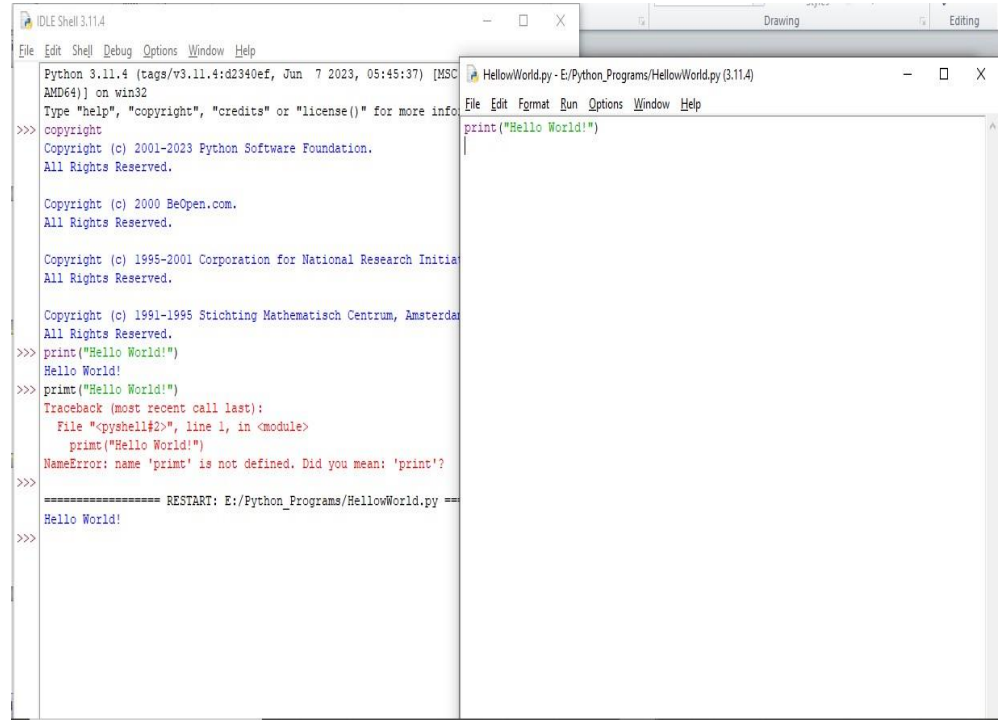
Copyright (c) 2000 BeOpen.com.
All Rights Reserved.

Copyright (c) 1995-2001 Corporation for National Research Initiatives.
All Rights Reserved.

Copyright (c) 1991-1995 Stichting Mathematisch Centrum, Amsterdam.
All Rights Reserved.
>>> print("Hello World!")
Hello World!
>>> print("Hello World!")
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    print("Hello World!")
NameError: name 'print' is not defined. Did you mean: 'print'?
>>>
```

Python IDLE : Development mode

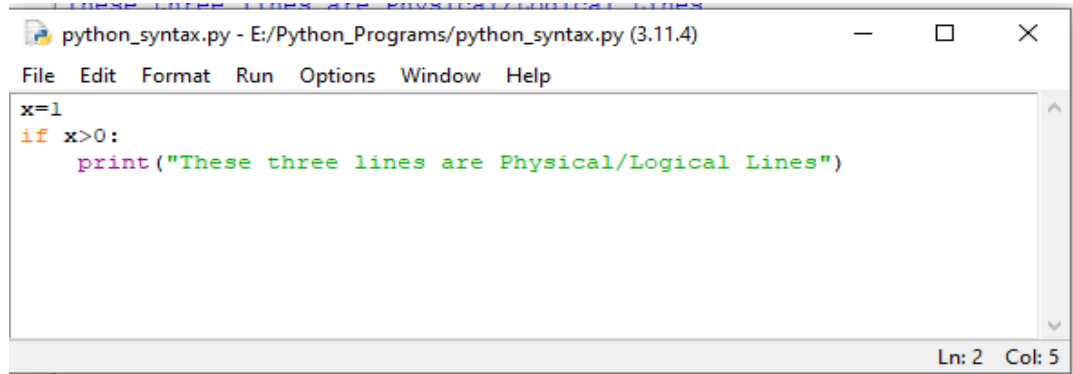
- At first, start with a new window.
- Clicking on "New window" under file menu a new window will come.
- Type **print "Hello World"** in the new window.
- Let's **save** (Save command is located under the File menu) the file now.
- We save the program as **helloworld.py** under **E:/python-programs** folder.
- To run the program select Run menu.
- Now click on Run Module or press **F5** as a shortcut key to see the result.



The screenshot displays two windows from the Python IDLE 3.11.4 application. The background window is the 'IDLE Shell 3.11.4' window, which shows the Python 3.11.4 startup screen with copyright information and a prompt. The foreground window is the 'HellowWorld.py - E:/Python_Programs/HellowWorld.py (3.11.4)' editor window. It contains a single line of Python code: `print("Hello World!")`. The shell window shows the output of this code: `Hello World!`. Below the output, a traceback message is visible: `NameError: name 'print' is not defined. Did you mean: 'print'?`. This suggests that the code was executed in a context where the `print` function was not available, possibly due to an incorrect file extension or a specific shell configuration.

Python Syntax

- A Python program is read by a parser. Python was designed to be a highly readable language. The syntax of the Python programming language is the set of rules which defines how a Python program will be written.
 - **Python Line Structure:**
 - A Python program is divided into a number of logical lines and every logical line is terminated by the token NEWLINE. A logical line is created from one or more physical lines.
 - A line contains only spaces, tabs, form feeds possibly a comment, is known as a blank line, and Python interpreter ignores it.
- A physical line is a sequence of characters terminated by an end-of-line sequence.



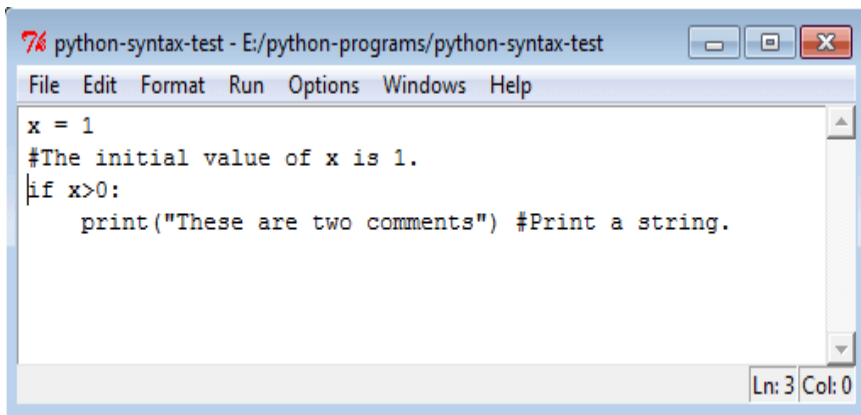
The screenshot shows a window titled 'python_syntax.py - E:/Python_Programs/python_syntax.py (3.11.4)'. The window contains a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code editor displays the following Python code:

```
x=1
if x>0:
    print("These three lines are Physical/Logical Lines")
```

The status bar at the bottom right indicates 'Ln: 2 Col: 5'.

Comments in Python:

- A comment begins with a hash character(#) which is not a part of the string literal and ends at the end of the physical line.
- All characters after the # character up to the end of the line are part of the comment and the Python interpreter ignores them.
- following example, It should be noted that Python has **no multi-lines or block comments** facility.



```
python-syntax-test - E:/python-programs/python-syntax-test
File Edit Format Run Options Windows Help
x = 1
#The initial value of x is 1.
if x>0:
    print("These are two comments") #Print a string.
Ln: 3 Col: 0
```

Since Python will ignore string literals that are not assigned to a variable, you can add a multiline string **(triple quotes)** in your code, and place your comment inside it:

```
"""
```

This is a comment
written in
more than just one line

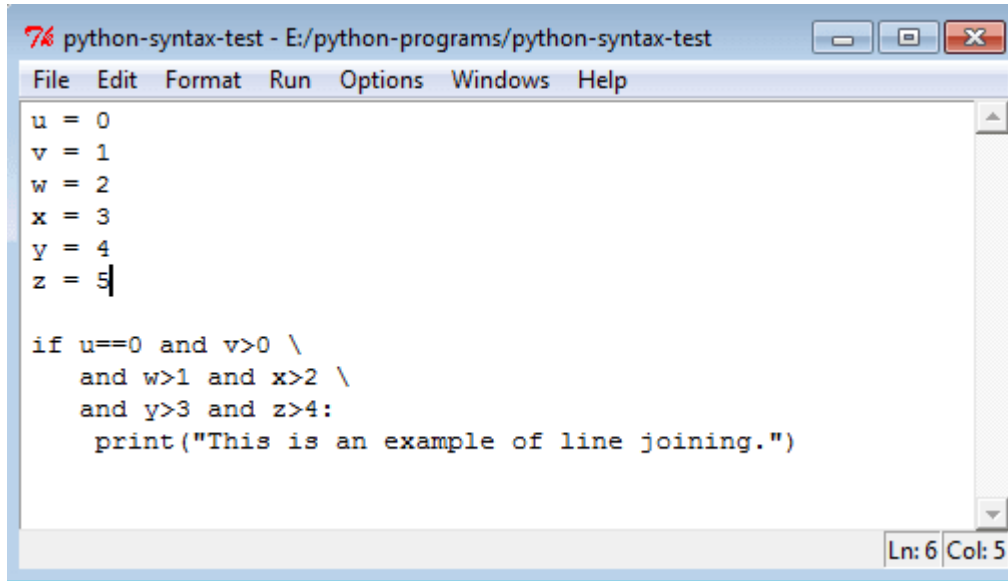
```
"""
```

```
print("Hello, World!")
```

As long as the string is not assigned to a variable, Python will read the code, but then ignore it, and you have made a **multiline comment**.

Joining two lines:

- When you want to write a long code in a single line you can break the logical line in two or more physical lines using backslash character(\).
- Therefore when a physical line ends with a backslash characters(\) and not a part of a string literal or comment then it can join another physical line.



The screenshot shows a window titled "python-syntax-test - E:/python-programs/python-syntax-test". The window contains a menu bar with "File", "Edit", "Format", "Run", "Options", "Windows", and "Help". The code editor displays the following Python code:

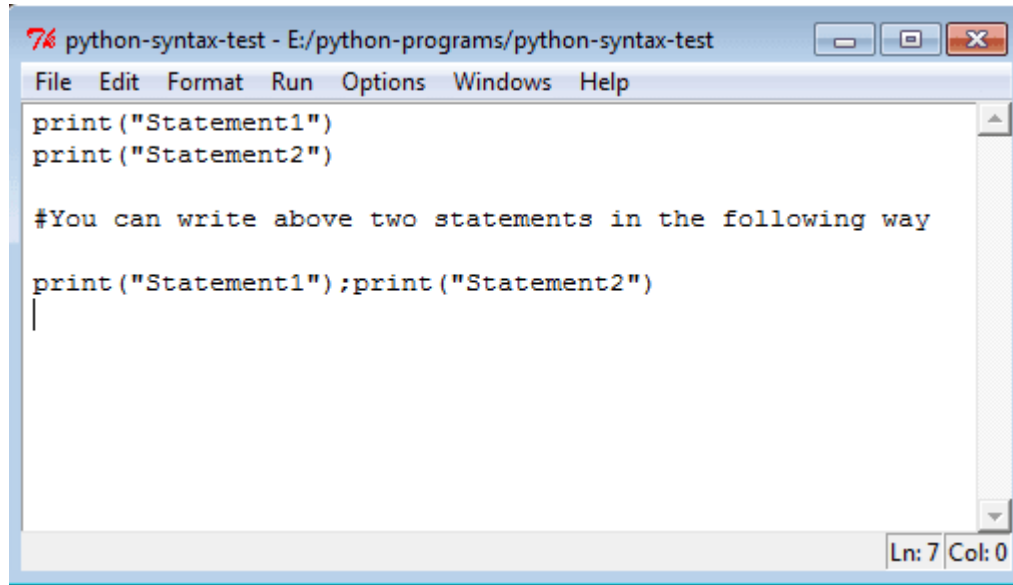
```
u = 0
v = 1
w = 2
x = 3
y = 4
z = 5

if u==0 and v>0 \
    and w>1 and x>2 \
    and y>3 and z>4:
    print("This is an example of line joining.")
```

The status bar at the bottom right indicates "Ln: 6 Col: 5".

Multiple Statements on a Single Line:

- You can write two separate statements into a single line using a **semicolon (;)** character between two line.



```
python-syntax-test - E:/python-programs/python-syntax-test
File Edit Format Run Options Windows Help
print("Statement1")
print("Statement2")

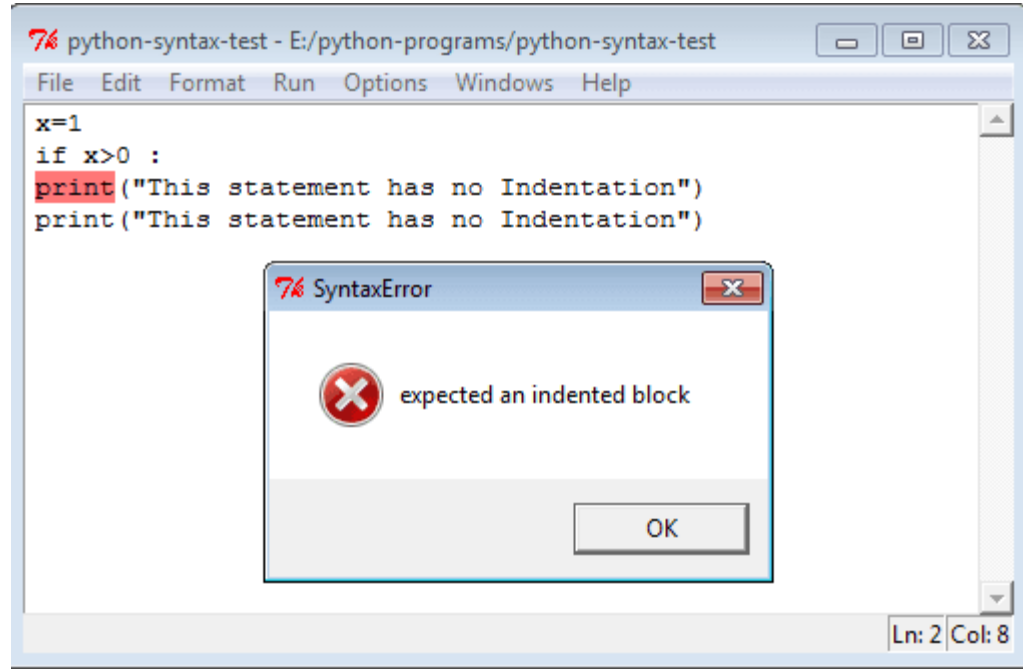
#You can write above two statements in the following way

print("Statement1");print("Statement2")
|
```

Ln: 7 Col: 0

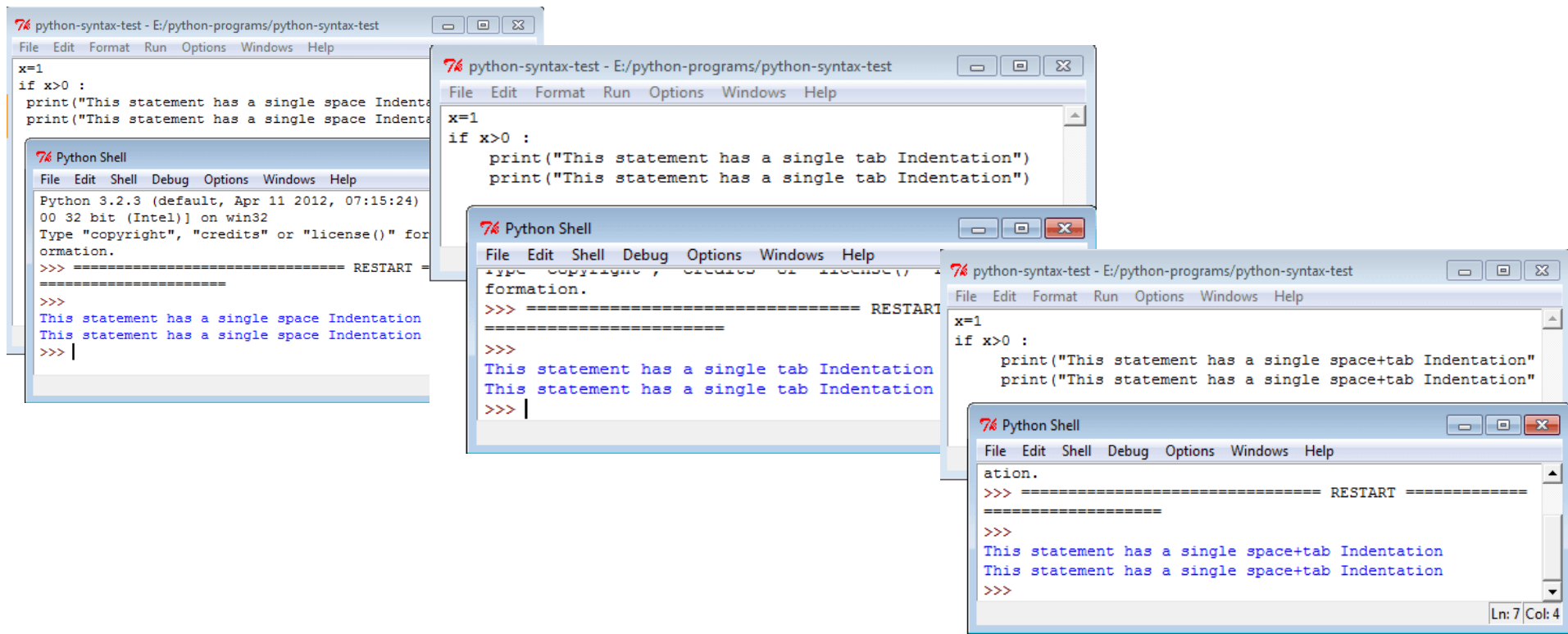
Indentation:

- Python uses **whitespace (spaces and tabs)** to define program blocks whereas other languages like C, C++ use braces (**{}**) to indicate blocks of codes for class, functions or flow control.
- **The number of whitespaces (spaces and tabs) in the indentation is not fixed, but all statements within the block must be the indented same amount.**
- In the following program, the block statements have no indentation.



Indentation:

- This is a program with **single space** indentation.
- This is a program with **single tab** indentation.
- Here is another program with an indentation of a **single space + a single tab**.



Python Coding Style:

- ☐ **Use 4 spaces** per indentation and no tabs.
- ☐ Do not mix tabs and spaces. Tabs create confusion and it is recommended to use only spaces.
- ☐ Maximum line length : 79 characters which help users with a small display.
- ☐ **Use blank lines** to separate top-level function and class definitions and single blank line to separate methods definitions inside a class and larger blocks of code inside functions.
- ☐ When possible, **put inline comments** (should be complete sentences).
- ☐ **Use spaces** around expressions and statements.

Python Reserve words

- The following identifiers are used as reserved words of the language, and **cannot be used as ordinary identifiers.**

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	el	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Python Variable

- **Variable and Value**

- A variable is a memory location where a programmer can store a value.

Example : roll_no, amount, name etc.

- Value is either string, numeric etc. Example : "Sara", 120, 25.36
- Variables are created when first assigned.
- Variables must be assigned before being referenced.
- The value stored in a variable can be accessed or updated later.
- **No declaration required**
- The type (string, int, float etc.) of the variable is determined by Python
- The interpreter allocates memory on the basis of the data type of a variable.

Python Variable Name Rules

- Must begin with a letter (a - z, A - B) or underscore (_)
- Other characters can be letters, numbers or _
- Case Sensitive
- Can be any (reasonable) length
- There are some reserved words which you cannot use as a variable name because Python uses them for other things.

Good Variable Name

- Choose meaningful name instead of short name. **roll_no is better than rn.**
- Maintain the length of a variable name. **Roll_no_of_a-student is too long.**
- Be consistent; roll_no or RollNo
- Begin a variable name with an **underscore(_)** character for a special case.

Python Assignment Statements

- The assignment statement creates new variables and gives them values. Basic assignment statement in Python is :

Syntax: **<variable> = <expr>**

- Where the equal sign (=) is used to assign value (right side) to a variable name (left side). See the following statements :
- One thing is important, assignment statement read right to left only.

```
1  >>> Item_name = "Computer" #A String
2  >>> Item_qty = 10 #An Integer
3  >>> Item_value = 1000.23 #A floating point
4  >>> print(Item_name)
5  Computer
6  >>> print(Item_qty)
7  10
8  >>> print(Item_value)
9  1000.23
10 >>>
```

Multiple Assignment

- The basic assignment statement works for a single variable and a single expression. You can also assign a single value to more than one variables simultaneously.

Syntax: var1=var2=var3...varn= <expr>

- **Example:** x = y = z = 1
- Now check the individual value in Python Shell.

1	>>> x = y = z = 1
2	>>> print(x)
3	1
4	>>> print(y)
5	1
6	>>> print(z)
7	1
8	>>>

Multiple Assignment

- An another assignment statement where the variables assign many values at the same time.

Syntax: <var>, <var>, ..., <var> = <expr>, <expr>, ..., <expr>

- **Example:**

x, y, z = 1, 2, "abcd"

- In the above example x, y and z simultaneously get the new values 1, 2 and "abcd".

1	>>> x,y,z = 1,2,"abcd"
2	>>> print(x)
3	1
4	>>> print(y)
5	2
6	>>> print(z)
7	abcd

Reuse variable names:

You can reuse variable names **by simply assigning a new value to them** :

1	>>> x = 100
2	>>> print(x)
3	100
4	>>> x = "Python"
5	>>> print(x)
6	Python
7	>>>

Swap variables

- Python swap values in a single line and this applies to all objects in python.

Syntax: `var1, var2 = var2, var1`

```
1  >>> x = 10
2  >>> y = 20
3  >>> print(x)
4  10
5  >>> print(y)
6  20
7  >>> x, y = y, x
8  >>> print(x)
9  20
10 >>> print(y)
11 10
12 >>>
```

Local and global variables in Python

- In Python, variables that are only referenced inside a function are implicitly global.
- If a variable is assigned a value anywhere within the function's body, it's assumed to be a local unless explicitly declared as global.

```
1 var1 = "Python"
2 def func1():
3     var1 = "PHP"
4     print("In side func1() var1 = ",var1)
5
6 def func2():
7     print("In side func2() var1 = ",var1)
8 func1()
9 func2()
```

Output:

```
In side func1() var1 =  PHP
In side func2() var1 =  Python
```

Local and global variables in Python

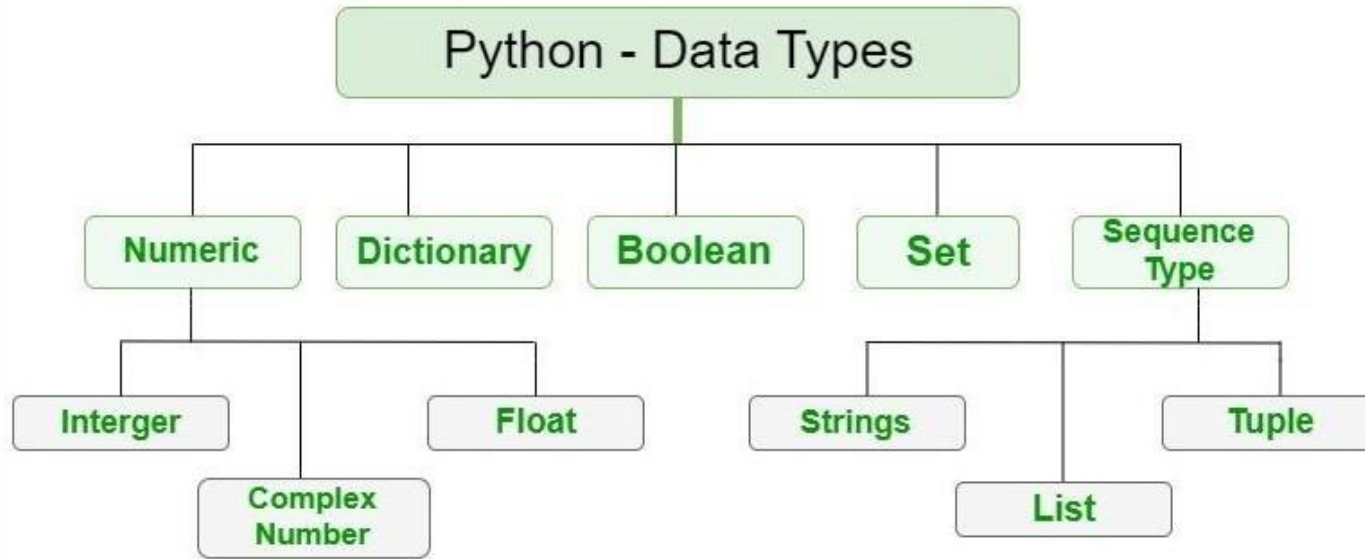
- You can use a global variable in other functions by declaring it as global keyword :

```
1 def func1():
2     global var1
3     var1 = "PHP"
4     print("In side func1() var1 = ",var1)
5
6 def func2():
7     print("In side func2() var1 = ",var1)
8 func1()
9 func2()
```

Output:

```
In side func1() var1 =  PHP
In side func2() var1 =  PHP
```

Python Data Types



Numbers

- Numbers are created by numeric literals. Numeric objects are **immutable**, which means when an object is created its value cannot be changed.
- **Python has three distinct numeric types:** integers, floating point numbers, and complex numbers.
- Integers represent negative and positive integers without fractional parts whereas floating point numbers represents negative and positive numbers with fractional parts.
- In addition, Booleans are a subtype of plain integers.

```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> a=1452
>>> type(a)
<class 'int'>
>>> b=(-4587)
>>> type(b)
<class 'int'>
>>> c=0
>>> type(c)
<class 'int'>
>>> g=1.03
>>> type(g)
<class 'float'>
>>> h=-11.23
>>> type(h)
<class 'float'>
>>> i=.34
>>> type(i)
<class 'float'>
>>> j=2.12e-10
>>> type(j)
<class 'float'>
>>> k=5E220
>>> type(k)
<class 'float'>
>>>
```

Ln: 27 Col: 4

Floats and Integers

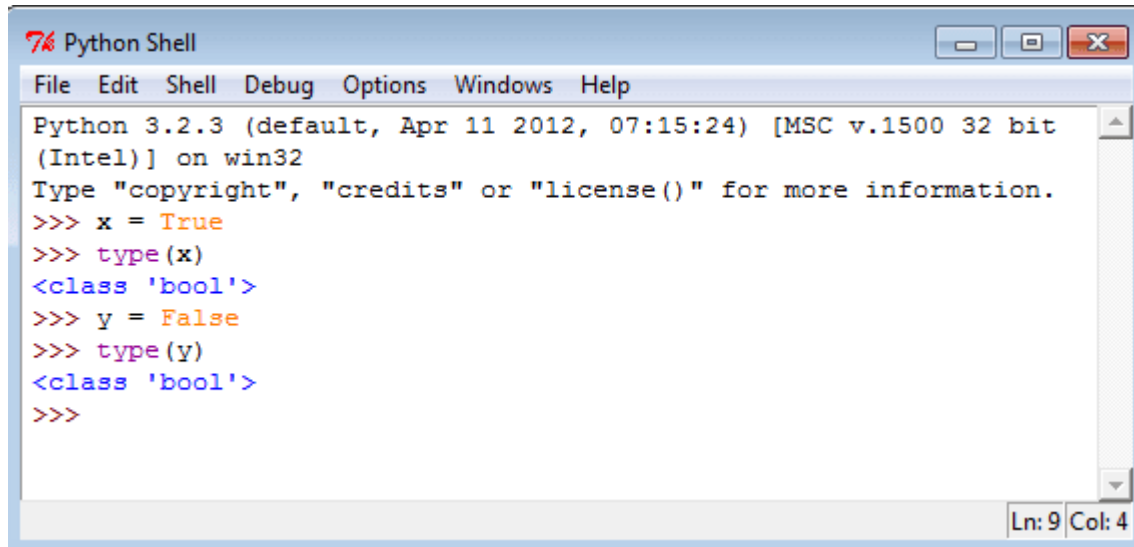
1	>>> x = 8
2	>>> y = 7
3	>>> x+y
4	>>> x-y
5	>>> x/y
6	>>> x*y

Output:

```
15
1
1.1428571428571428
56
```

Boolean (bool)

- The simplest build-in type in Python is the bool type, it represents the truth values False and True.
- See the following statements in Python shell.

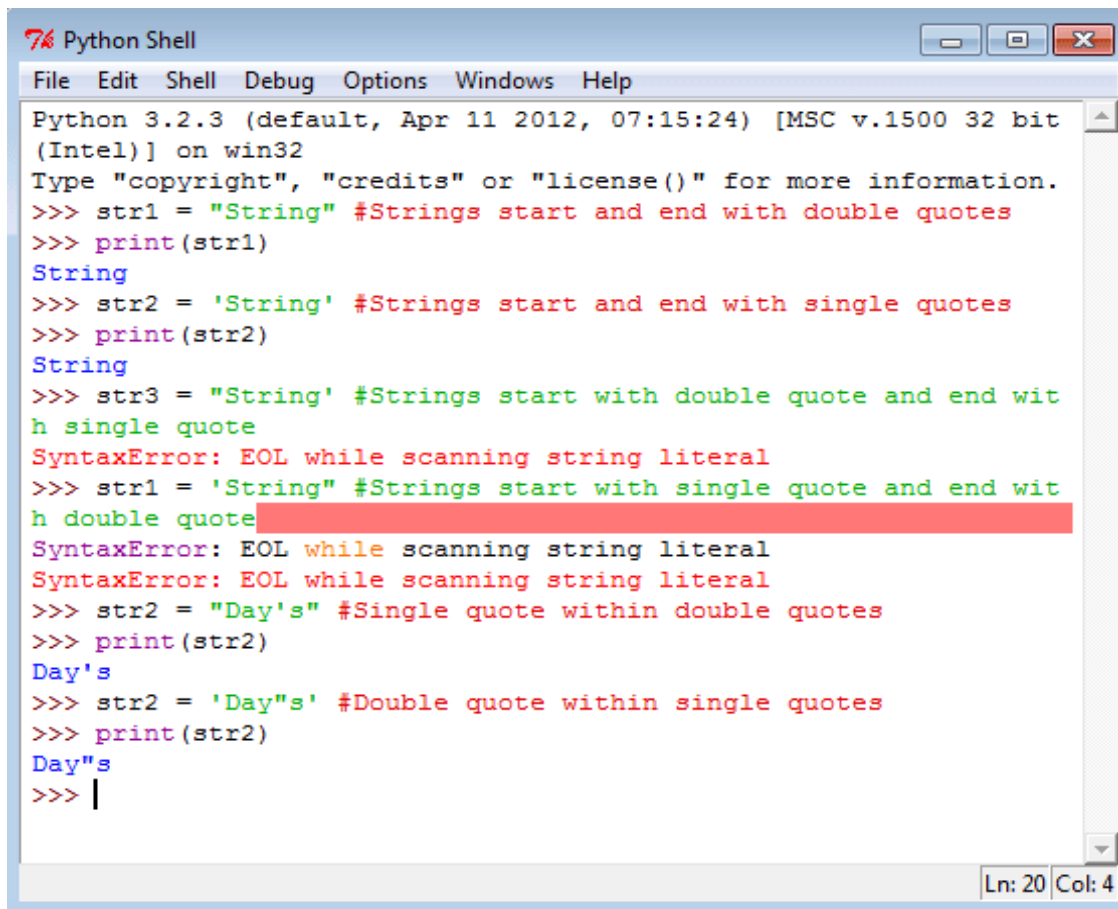


```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> x = True
>>> type(x)
<class 'bool'>
>>> y = False
>>> type(y)
<class 'bool'>
>>>
```

Ln: 9 Col: 4

Strings

- In Python, a string type object is a sequence (left-to- right order) of characters.
- Strings start and end with single or double quotes Python strings are **immutable**.
- Single and double quoted strings are same and you can use a single quote within a string when it is surrounded by double quote and vice versa.
- Declaring a string is simple, see the following statements.

A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> str1 = "String" #Strings start and end with double quotes
>>> print(str1)
String
>>> str2 = 'String' #Strings start and end with single quotes
>>> print(str2)
String
>>> str3 = "String' #Strings start with double quote and end wit
h single quote
SyntaxError: EOL while scanning string literal
>>> str1 = 'String" #Strings start with single quote and end wit
h double quote
SyntaxError: EOL while scanning string literal
SyntaxError: EOL while scanning string literal
>>> str2 = "Day's" #Single quote within double quotes
>>> print(str2)
Day's
>>> str2 = 'Day"s' #Double quote within single quotes
>>> print(str2)
Day"s
>>> |
```

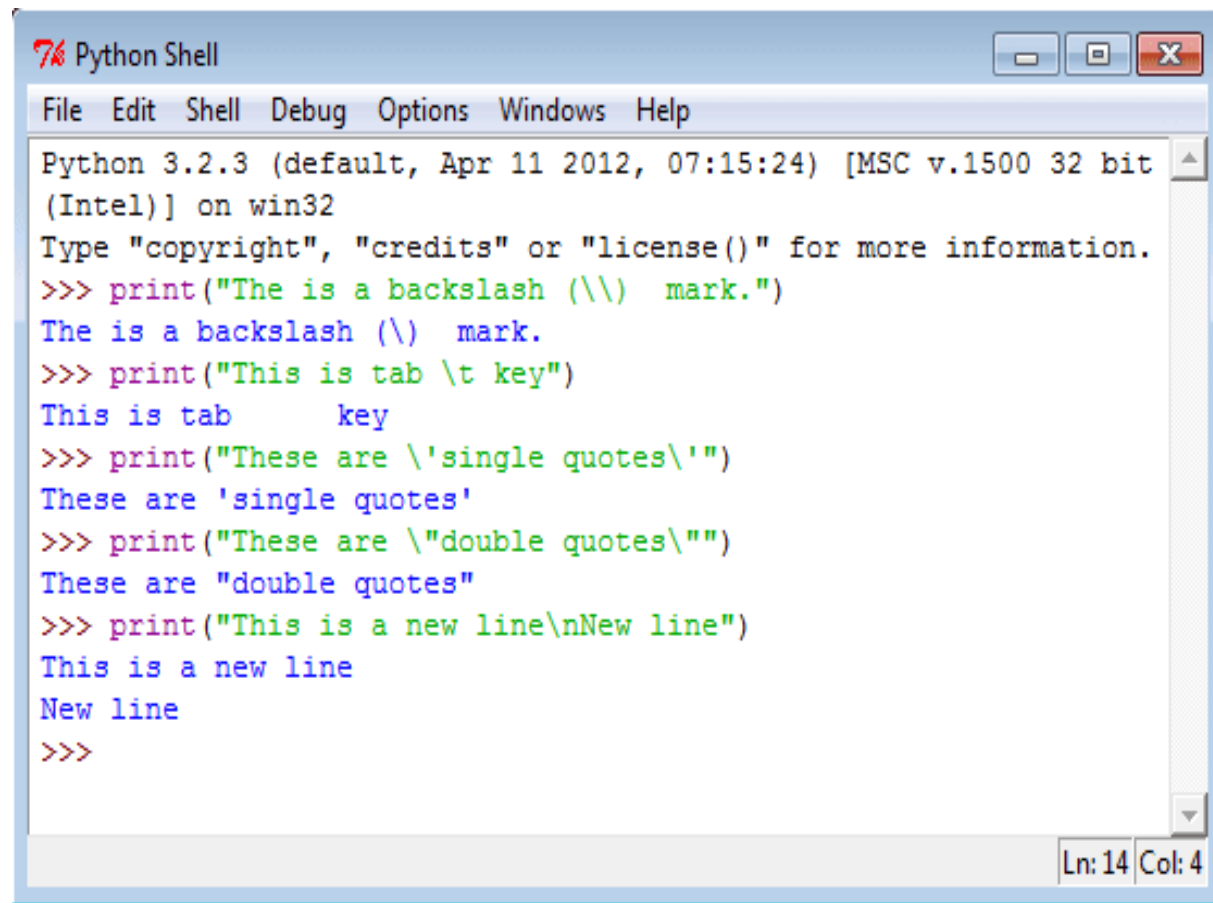
The status bar at the bottom right shows "Ln: 20 Col: 4".

Special characters in strings

- The backslash (\) character is used to introduce a special character. See the following table.

Escape sequence	Meaning
<code>\n</code>	Newline
<code>\t</code>	Horizontal Tab
<code>\\</code>	Backslash
<code>\'</code>	Single Quote
<code>\"</code>	Double Quote

Special characters in strings

A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following text:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit  
(Intel)] on win32  
Type "copyright", "credits" or "license()" for more information.  
>>> print("The is a backslash (\\) mark.")  
The is a backslash (\) mark.  
>>> print("This is tab \t key")  
This is tab      key  
>>> print("These are \'single quotes\'")  
These are 'single quotes'  
>>> print("These are \"double quotes\"")  
These are "double quotes"  
>>> print("This is a new line\nNew line")  
This is a new line  
New line  
>>>
```

The status bar at the bottom right shows "Ln: 14 Col: 4".

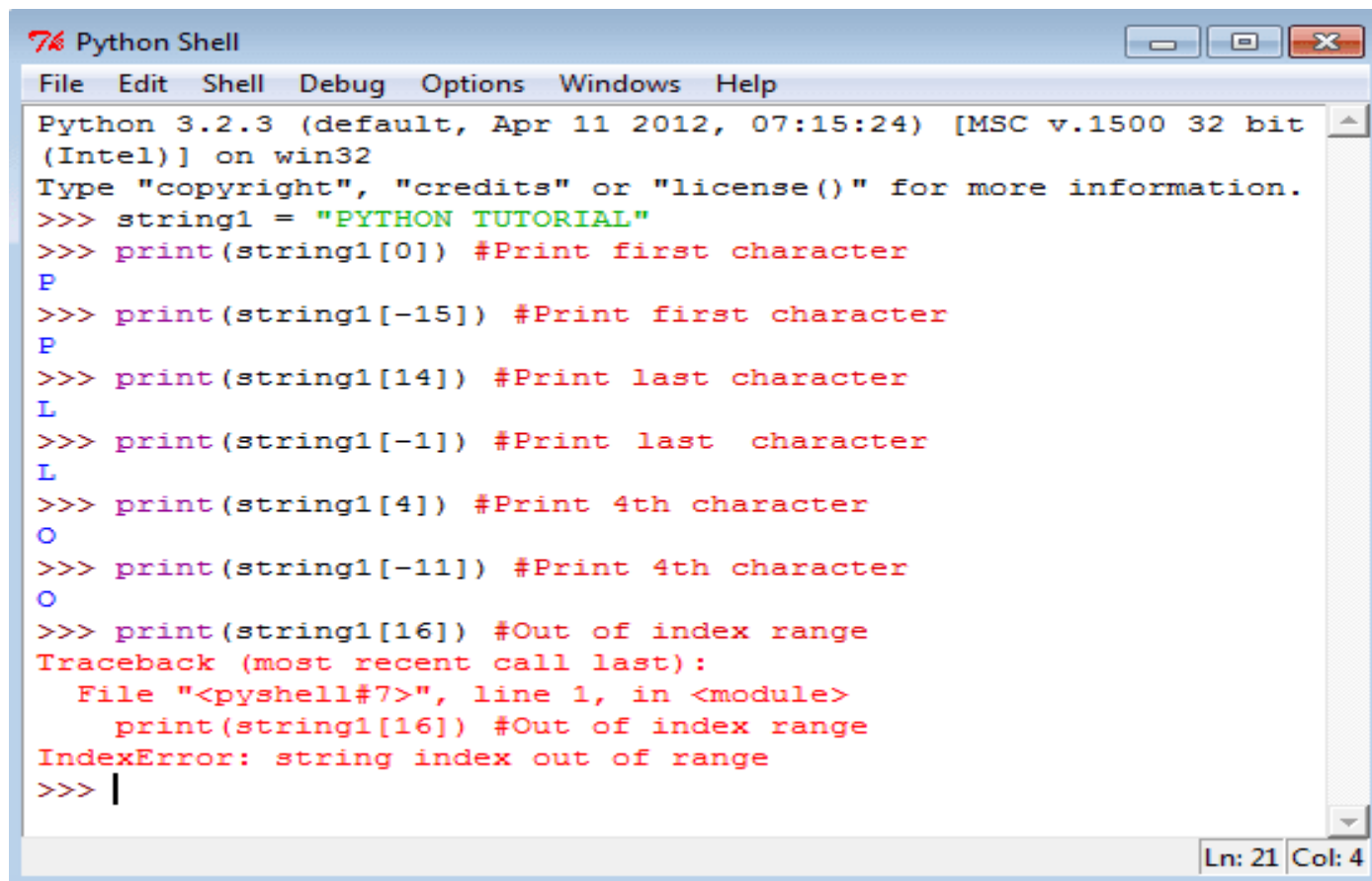
String index and accessing string elements

- Strings are **arrays of characters** and elements of an array **can be accessed using indexing**.
- Index start with **0 from left side** and **-1** when starting **from right side**.

string1 = "PYTHON TUTORIAL"

Character	P	Y	T	H	O	N		T	U	T	O	R	I	A	L
Index (from left)	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Index (from right)	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

See the following statements to access single character from various positions.

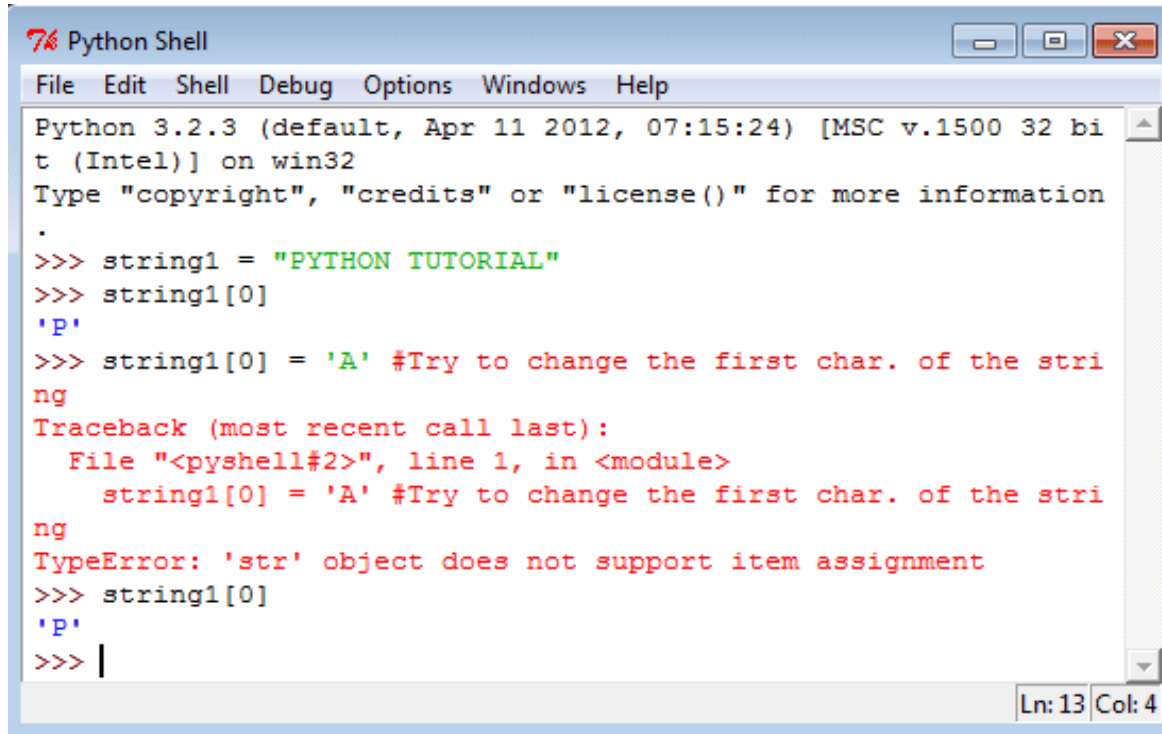
A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> string1 = "PYTHON TUTORIAL"
>>> print(string1[0]) #Print first character
P
>>> print(string1[-15]) #Print first character
P
>>> print(string1[14]) #Print last character
L
>>> print(string1[-1]) #Print last character
L
>>> print(string1[4]) #Print 4th character
O
>>> print(string1[-11]) #Print 4th character
O
>>> print(string1[16]) #Out of index range
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    print(string1[16]) #Out of index range
IndexError: string index out of range
>>> |
```

The status bar at the bottom right shows "Ln: 21 Col: 4".

Strings are immutable

- Strings are **immutable** character sets. Once a string is generated, you can not change any character within the string. See the following statements.

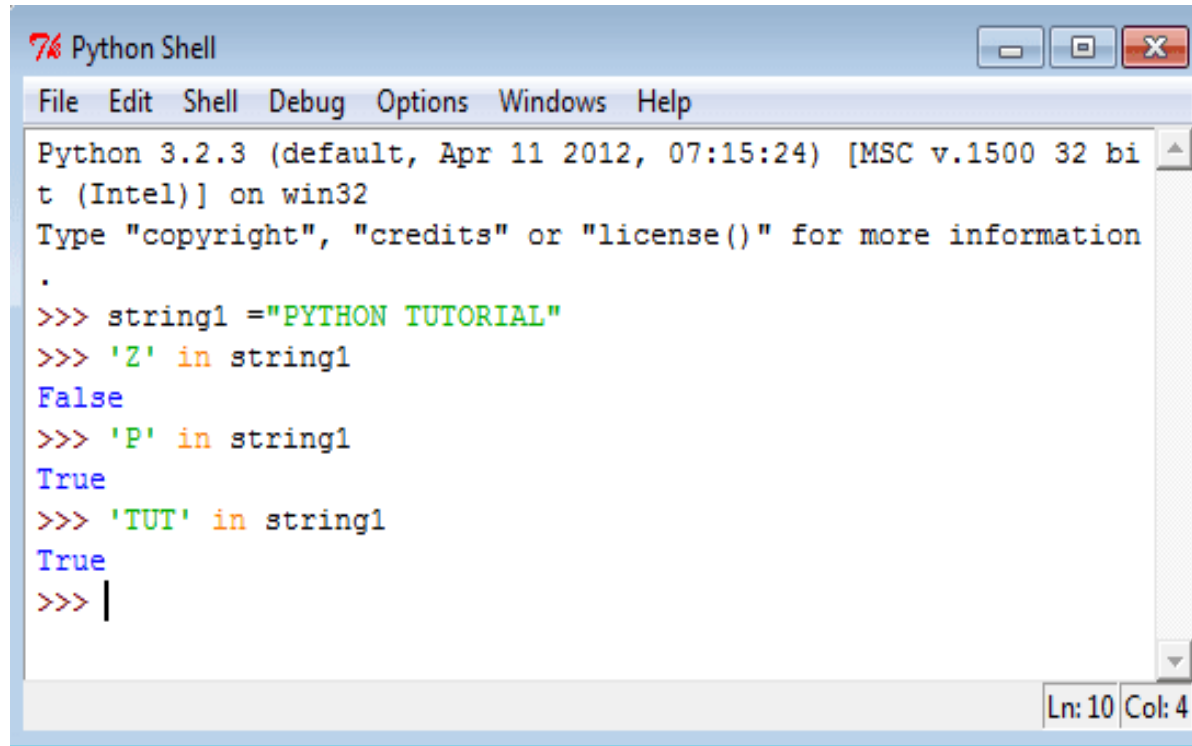


```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bi
t (Intel)] on win32
Type "copyright", "credits" or "license()" for more information
.
>>> string1 = "PYTHON TUTORIAL"
>>> string1[0]
'P'
>>> string1[0] = 'A' #Try to change the first char. of the stri
ng
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    string1[0] = 'A' #Try to change the first char. of the stri
ng
TypeError: 'str' object does not support item assignment
>>> string1[0]
'P'
>>> |
```

Ln: 13 Col: 4

'in' operator in Strings

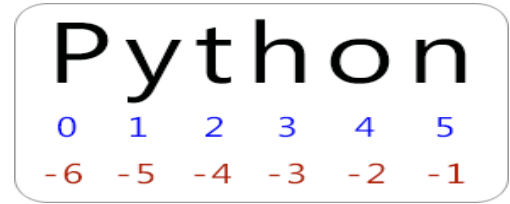
- The 'in' operator is used to check whether a **character** or a **substring** is present in a string or not. **The expression returns a Boolean value.**

A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following content: "Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32", "Type 'copyright', 'credits' or 'license()' for more information", a prompt ".", and three code snippets: ">>> string1 = 'PYTHON TUTORIAL'", ">>> 'Z' in string1" followed by "False", and ">>> 'P' in string1" followed by "True". The cursor is at the end of the third snippet. The status bar at the bottom right shows "Ln: 10 Col: 4".

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information
.
>>> string1 = "PYTHON TUTORIAL"
>>> 'Z' in string1
False
>>> 'P' in string1
True
>>> 'TUT' in string1
True
>>> |
```

String Slicing

- To cut a substring from a string is called string slicing.
- Here two indices are used separated by a colon [start : end + 1].
- A slice 3:6 means index characters of 3rd, 4th, and 5th positions.
- The second integer index i.e. 6 is not included.
- You can use negative indices for slicing. See the following statements.



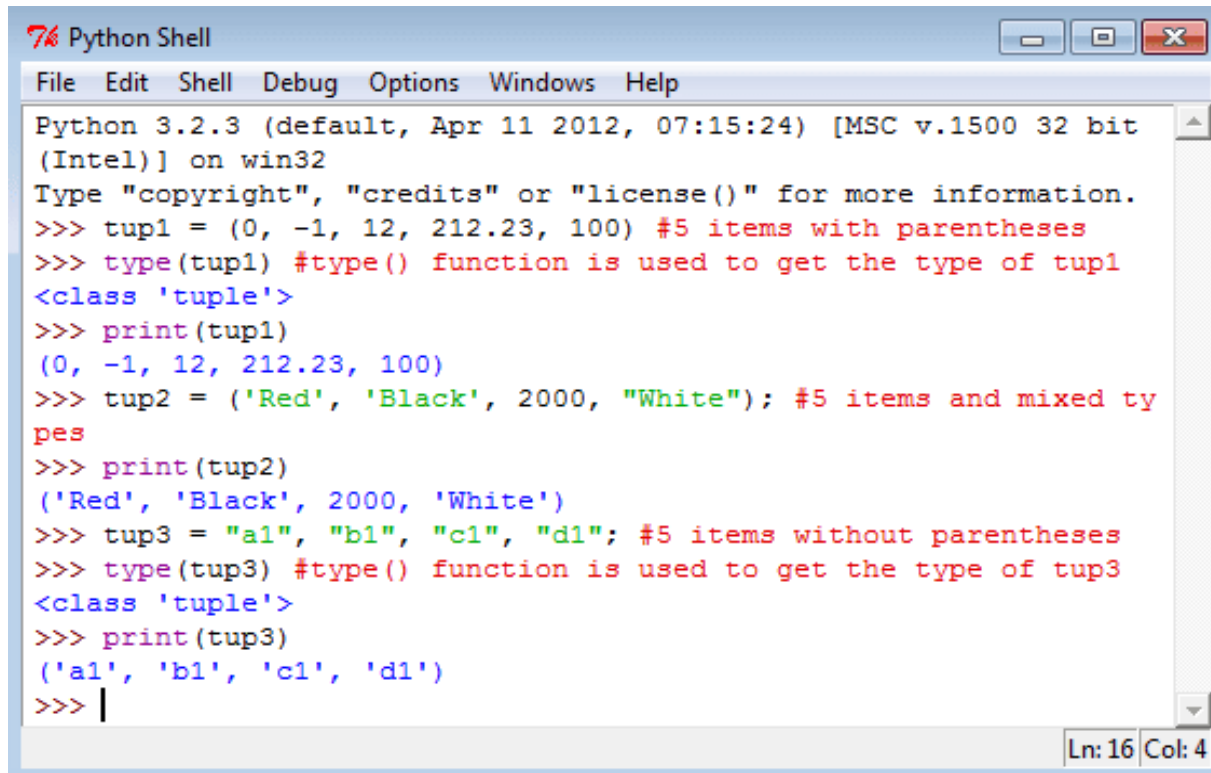
```
IDLE Shell 3.13.2
File Edit Shell Debug Options Window Help
Python 3.13.2 (tags/v3.13.2:1
AMD64)] on win32
Type "help", "copyright", "c:
>>> str1="python"
>>> print(str1[3:6])
hon
>>> print(str1[-4:-1])
tho
>>> print(str1[-4:])
thon
```

Conversion

	Output
>>> float("4.5")	4.5
>>> int("25")	25
>>> int(5.625)	5
>>> float(6)	6.0
>>> int(True)	1
>>> float(False)	0.0
>>> str(True)	'True'
>>> bool(0)	False
>>> bool('Hello world')	True
>>> bool(223.5)	True

Tuple

- A tuple is a **container** which holds a series of **comma-separated values** (items or elements) **between parentheses**.
- Tuples are **immutable** (i.e. you cannot change its content once created) and **can hold mix data types**.

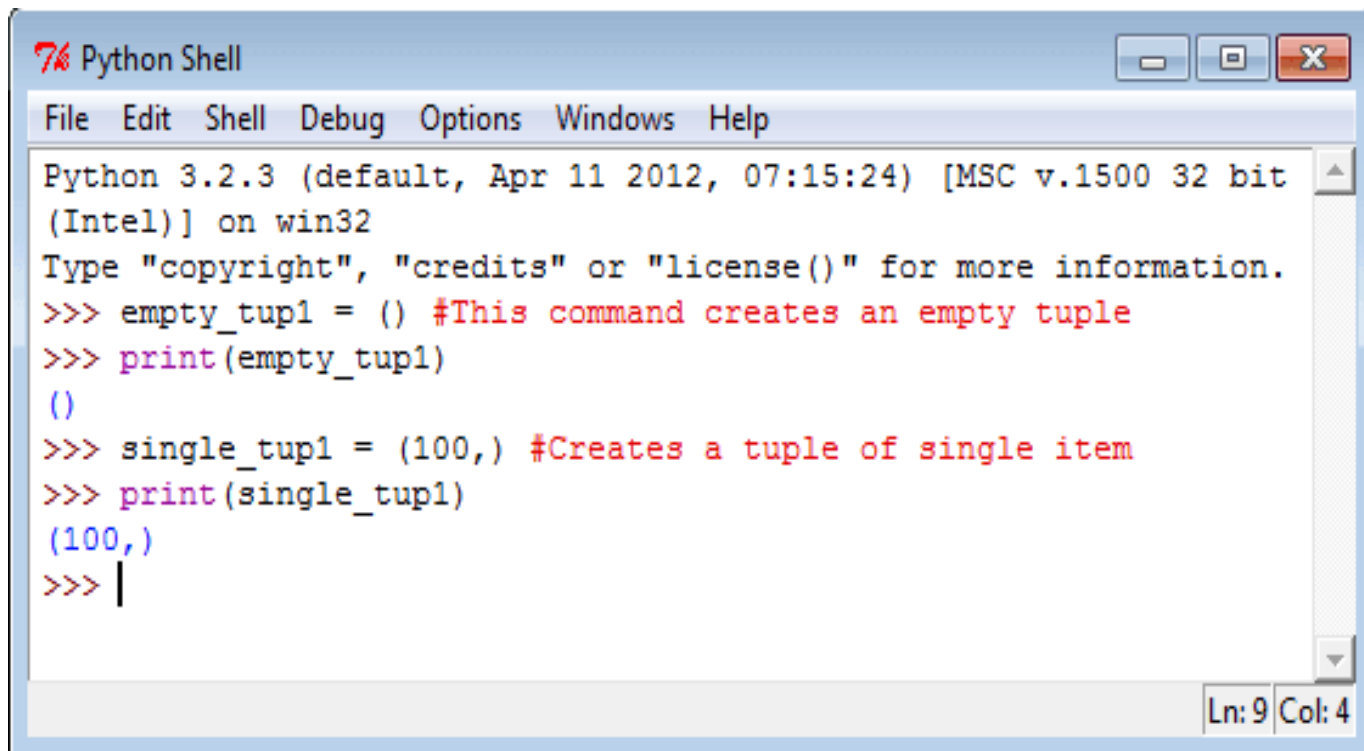
A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following Python code and its output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> tup1 = (0, -1, 12, 212.23, 100) #5 items with parentheses
>>> type(tup1) #type() function is used to get the type of tup1
<class 'tuple'>
>>> print(tup1)
(0, -1, 12, 212.23, 100)
>>> tup2 = ('Red', 'Black', 2000, "White"); #5 items and mixed types
>>> print(tup2)
('Red', 'Black', 2000, 'White')
>>> tup3 = "a1", "b1", "c1", "d1"; #5 items without parentheses
>>> type(tup3) #type() function is used to get the type of tup3
<class 'tuple'>
>>> print(tup3)
('a1', 'b1', 'c1', 'd1')
>>> |
```

The status bar at the bottom right shows "Ln: 16 Col: 4".

Tuple

- To create an **empty tuple** or **create a tuple with single element** use the following commands.

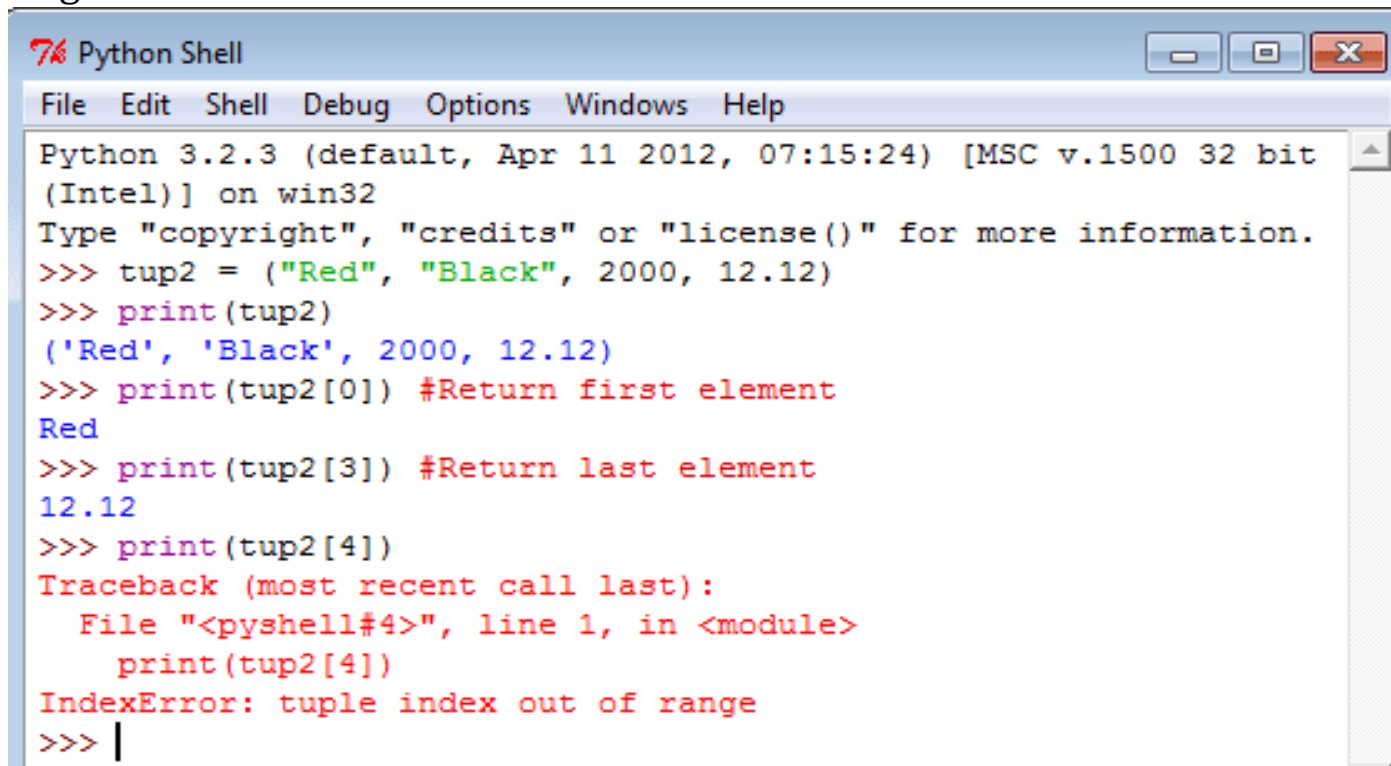


```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> empty_tup1 = () #This command creates an empty tuple
>>> print(empty_tup1)
()
>>> single_tup1 = (100,) #Creates a tuple of single item
>>> print(single_tup1)
(100,)
>>> |
```

Ln: 9 Col: 4

Tuple

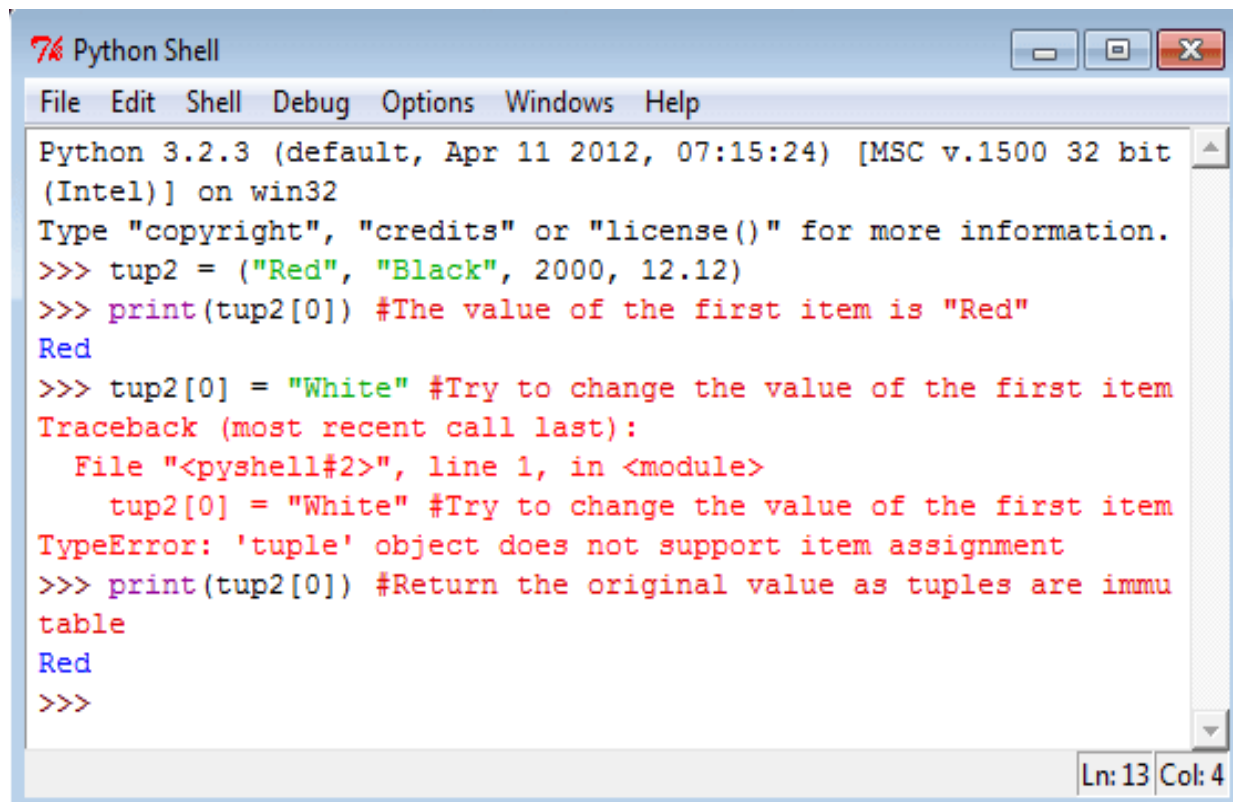
- Elements of a tuple are indexed like other sequences. **The tuple indices start at 0.** See the following statements.



```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> tup2 = ("Red", "Black", 2000, 12.12)
>>> print(tup2)
('Red', 'Black', 2000, 12.12)
>>> print(tup2[0]) #Return first element
Red
>>> print(tup2[3]) #Return last element
12.12
>>> print(tup2[4])
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    print(tup2[4])
IndexError: tuple index out of range
>>> |
```

Tuple

- Tuples are immutable which means it's items values are unchangeable.

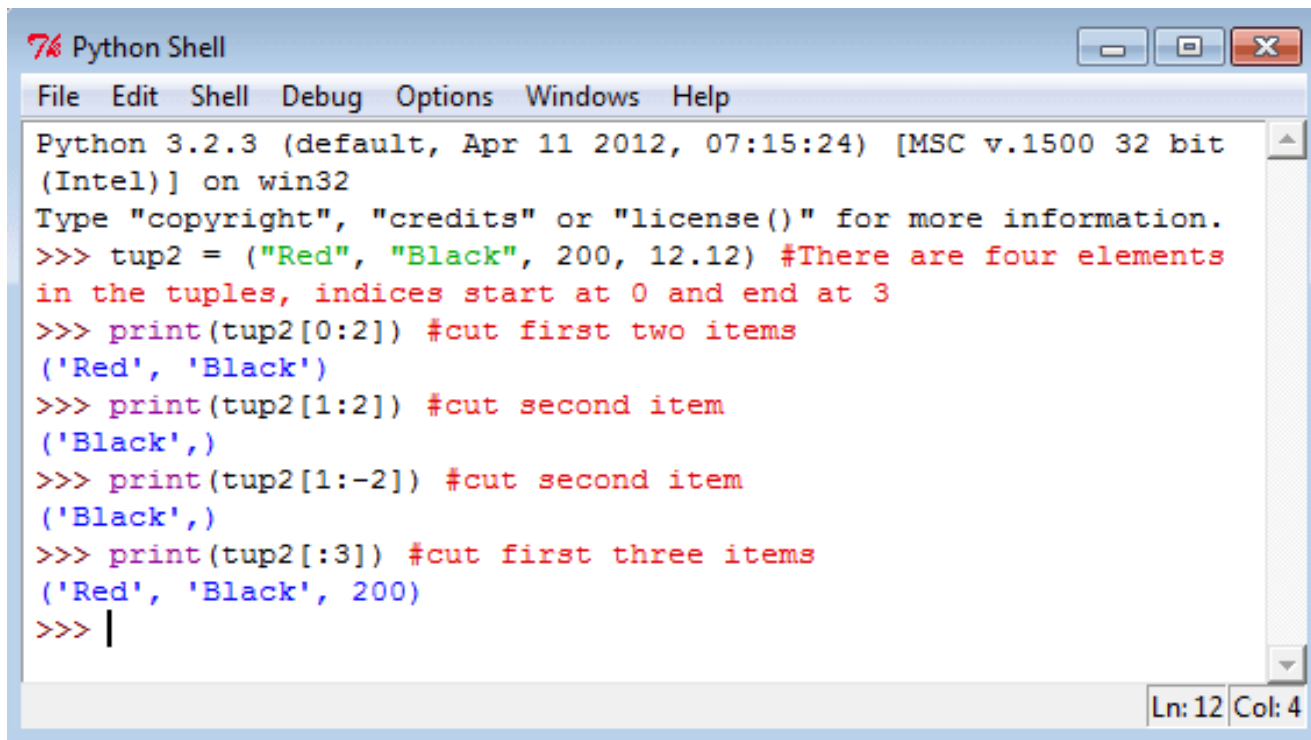


```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> tup2 = ("Red", "Black", 2000, 12.12)
>>> print(tup2[0]) #The value of the first item is "Red"
Red
>>> tup2[0] = "White" #Try to change the value of the first item
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    tup2[0] = "White" #Try to change the value of the first item
TypeError: 'tuple' object does not support item assignment
>>> print(tup2[0]) #Return the original value as tuples are immu
table
Red
>>>
```

Ln: 13 Col: 4

Slicing a tuple

- Like other sequences like strings, tuples can be sliced.
- Slicing a tuple creates a new tuple but it does not change the original tuple.

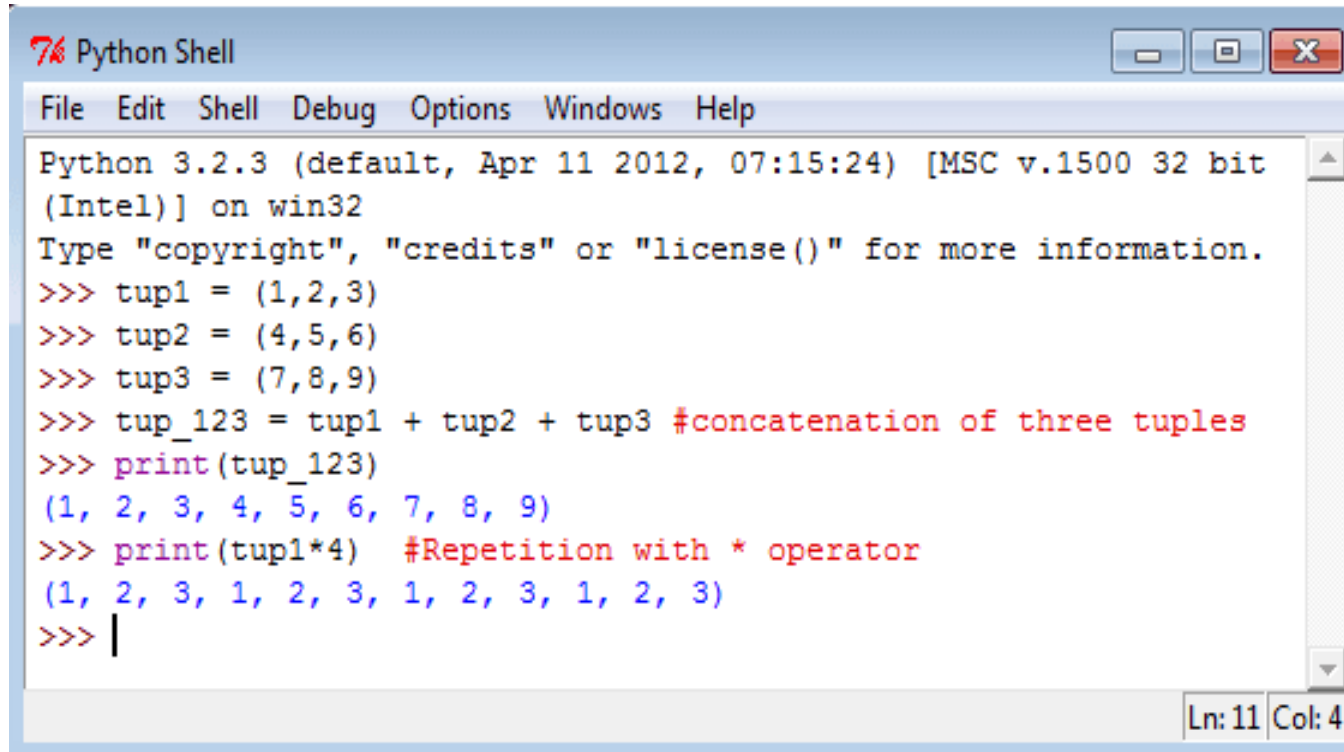


```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> tup2 = ("Red", "Black", 200, 12.12) #There are four elements
in the tuples, indices start at 0 and end at 3
>>> print(tup2[0:2]) #cut first two items
('Red', 'Black')
>>> print(tup2[1:2]) #cut second item
('Black',)
>>> print(tup2[1:-2]) #cut second item
('Black',)
>>> print(tup2[:3]) #cut first three items
('Red', 'Black', 200)
>>> |
```

Ln: 12 Col: 4

Using + and * operators in Tuple

- Use + operator to create a new tuple that is a **concatenation** of tuples and
- use * operator to repeat a tuple.

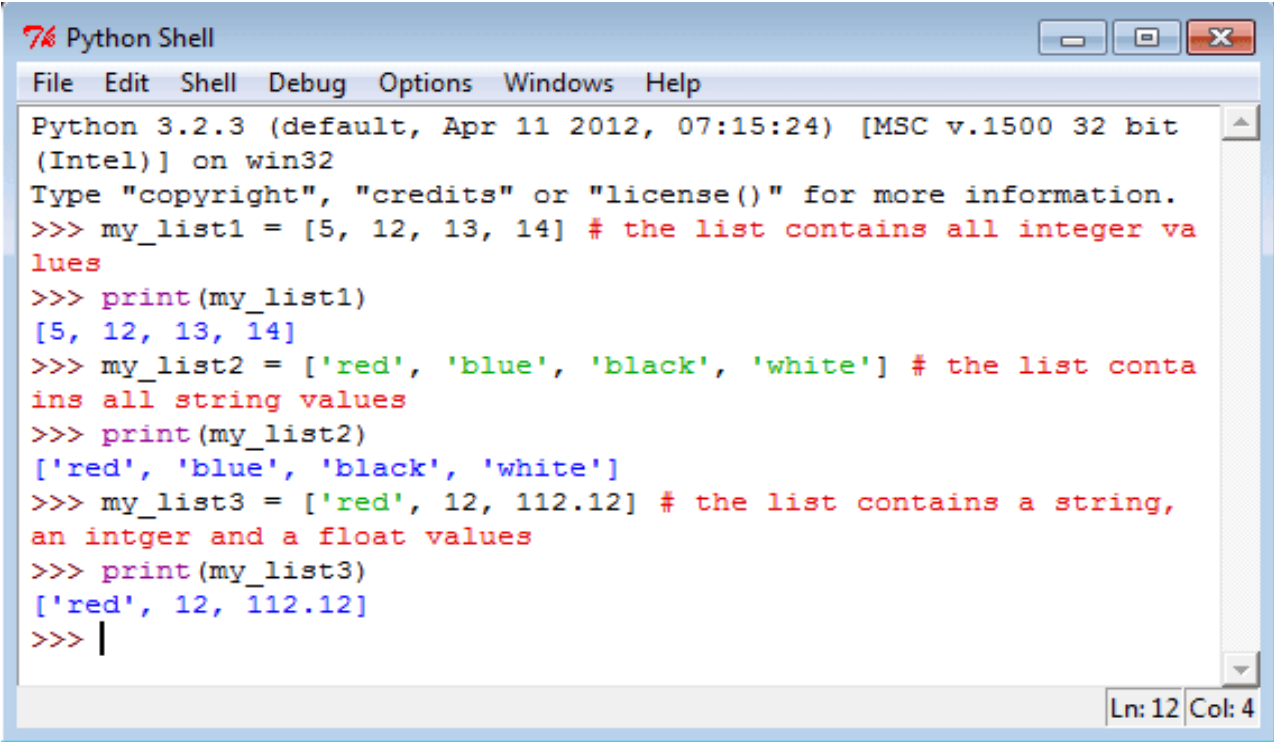
A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> tup1 = (1,2,3)
>>> tup2 = (4,5,6)
>>> tup3 = (7,8,9)
>>> tup_123 = tup1 + tup2 + tup3 #concatenation of three tuples
>>> print(tup_123)
(1, 2, 3, 4, 5, 6, 7, 8, 9)
>>> print(tup1*4) #Repetition with * operator
(1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3)
>>> |
```

The status bar at the bottom right shows "Ln: 11 Col: 4".

List - Creating List

- A list is a container which holds comma-separated values (items or elements) between **square brackets** where Items or **elements need not all have the same type**.

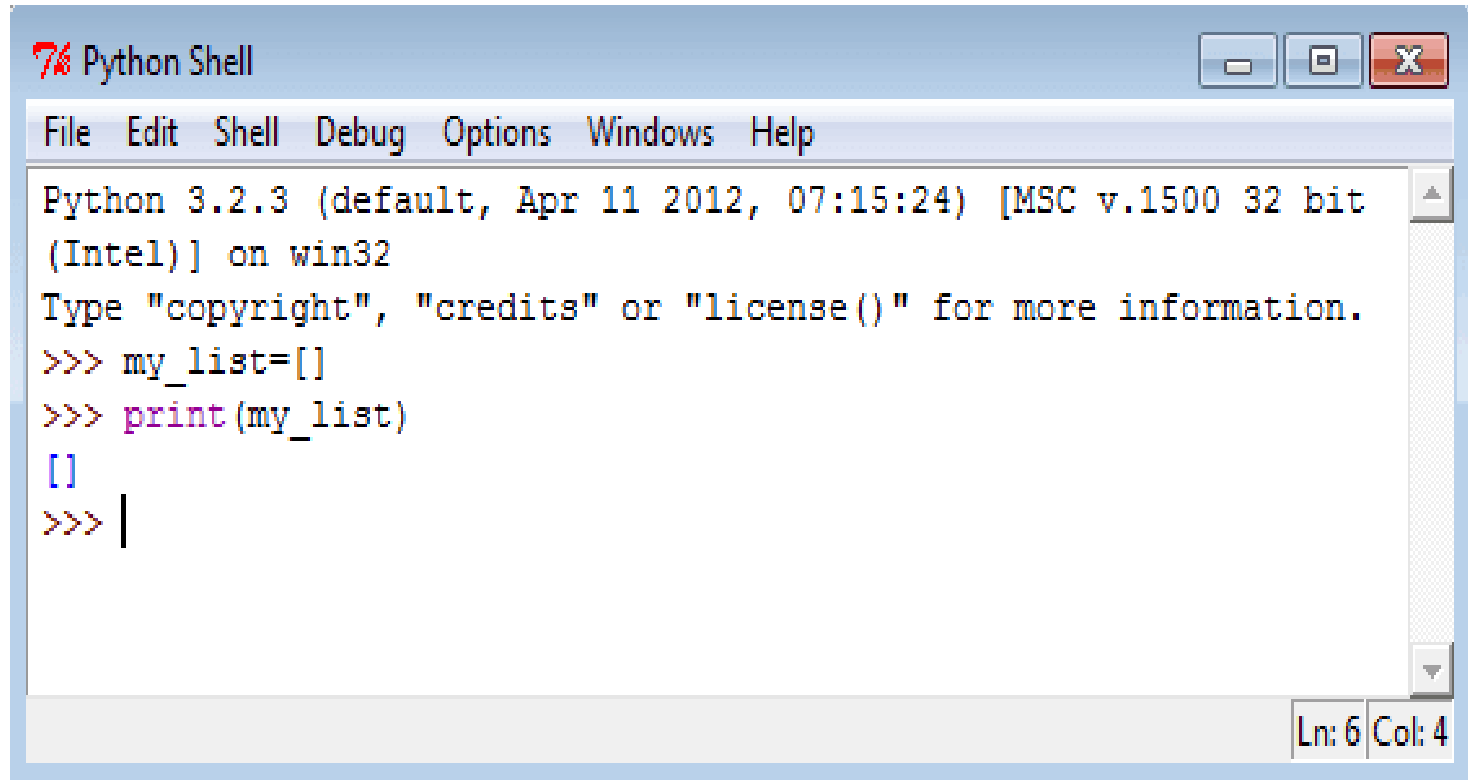
A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> my_list1 = [5, 12, 13, 14] # the list contains all integer va
lues
>>> print(my_list1)
[5, 12, 13, 14]
>>> my_list2 = ['red', 'blue', 'black', 'white'] # the list conta
ins all string values
>>> print(my_list2)
['red', 'blue', 'black', 'white']
>>> my_list3 = ['red', 12, 112.12] # the list contains a string,
an intger and a float values
>>> print(my_list3)
['red', 12, 112.12]
>>> |
```

The status bar at the bottom right shows "Ln: 12 Col: 4".

List - Creating List

- A list without any element is called an empty list. See the following statements.



The image shows a screenshot of a Python Shell window. The title bar reads "Python Shell" with standard window controls (minimize, maximize, close). The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area displays the following text:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit  
(Intel)] on win32  
Type "copyright", "credits" or "license()" for more information.  
>>> my_list=[]  
>>> print(my_list)  
[]  
>>> |
```

The status bar at the bottom right indicates "Ln: 6 Col: 4".

List indices

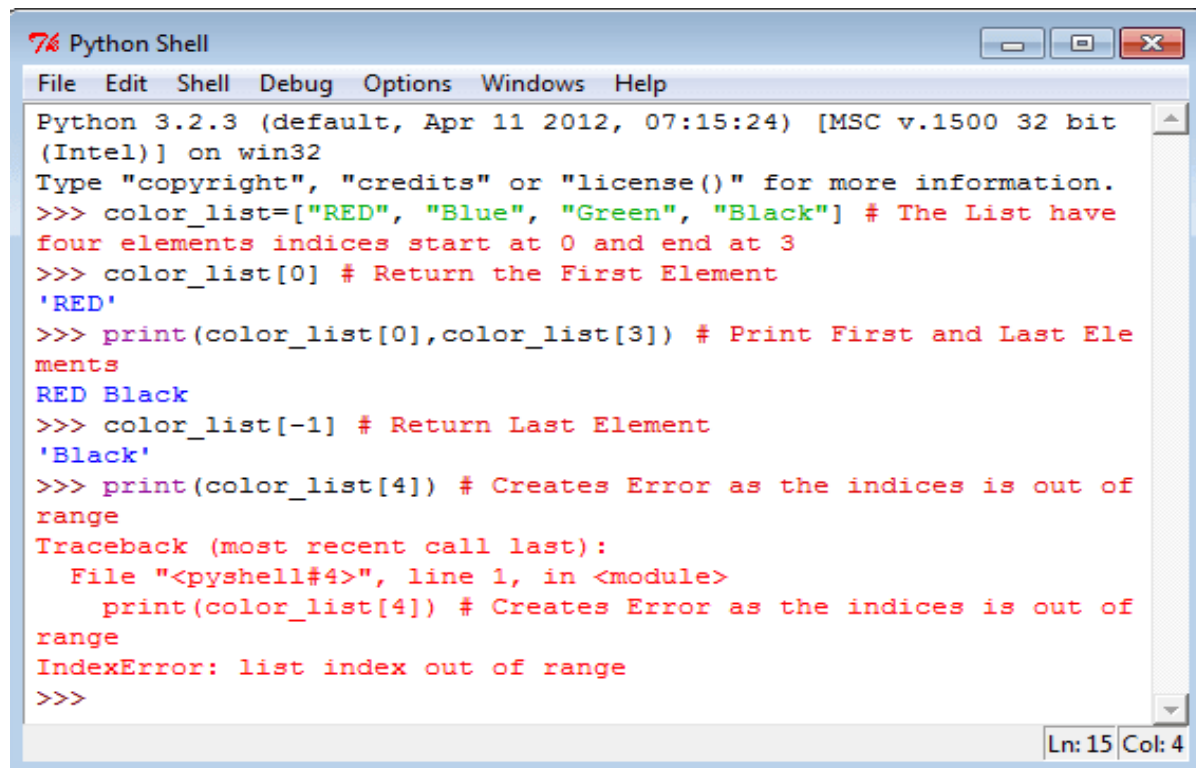
- List indices work the same way as string indices, list indices start at 0.
- If an index has a positive value it counts from the beginning and similarly it counts backward if the index has a negative value.
- As positive integers are used to index from the left end and negative integers are used to index from the right end, so every item of a list gives two alternatives indices.
- Let create a list called **color_list** with four items.

color_list=["RED", "Blue", "Green", "Black"]

Item	RED	Blue	Green	Black
Index (from left)	0	1	2	3
Index (from right)	-4	-3	-2	-1

List indices

- If you give any index value which is out of range then interpreter creates an error message. See the following statements.

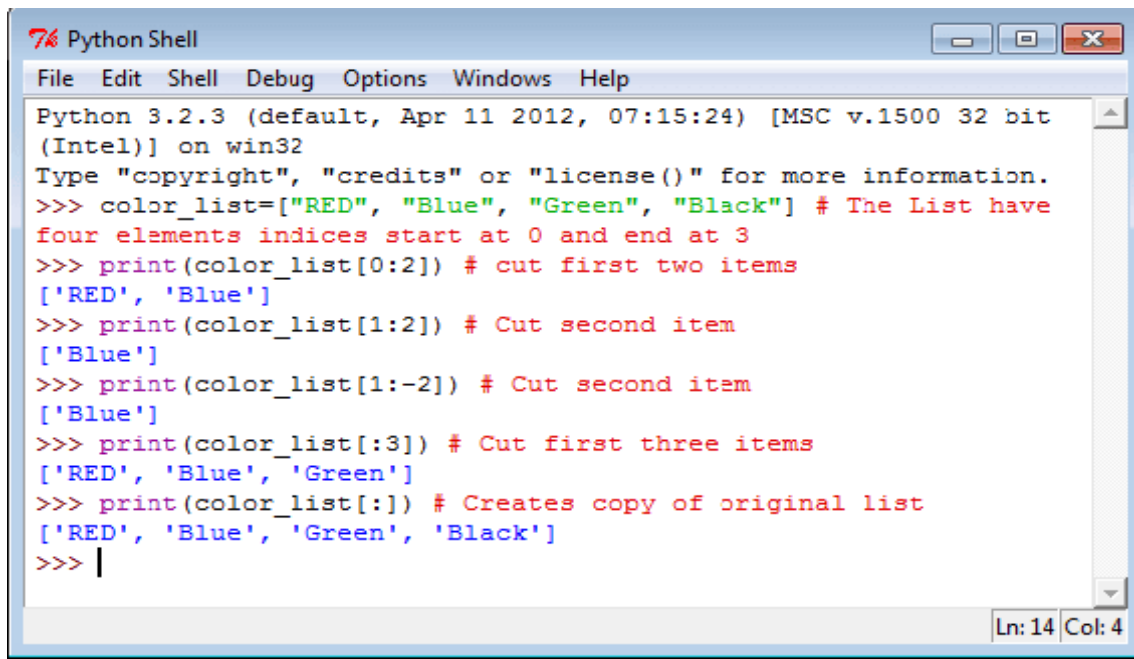
A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> color_list=["RED", "Blue", "Green", "Black"] # The List have four elements indices start at 0 and end at 3
>>> color_list[0] # Return the First Element
'RED'
>>> print(color_list[0],color_list[3]) # Print First and Last Elements
RED Black
>>> color_list[-1] # Return Last Element
'Black'
>>> print(color_list[4]) # Creates Error as the indices is out of range
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    print(color_list[4]) # Creates Error as the indices is out of range
IndexError: list index out of range
>>>
```

The status bar at the bottom right shows "Ln: 15 Col: 4".

List Slice

- Lists can be sliced like strings and other sequences. The syntax of list slices is easy :
sliced_list=List_Name[startIndex : endIndex]
- This refers to the items of a list starting at index startIndex and stopping just before index endIndex.
- The default values for list are 0 (startIndex) and the end (endIndex) of the list.
- If you omit both indices, the slice makes a copy of the original list.

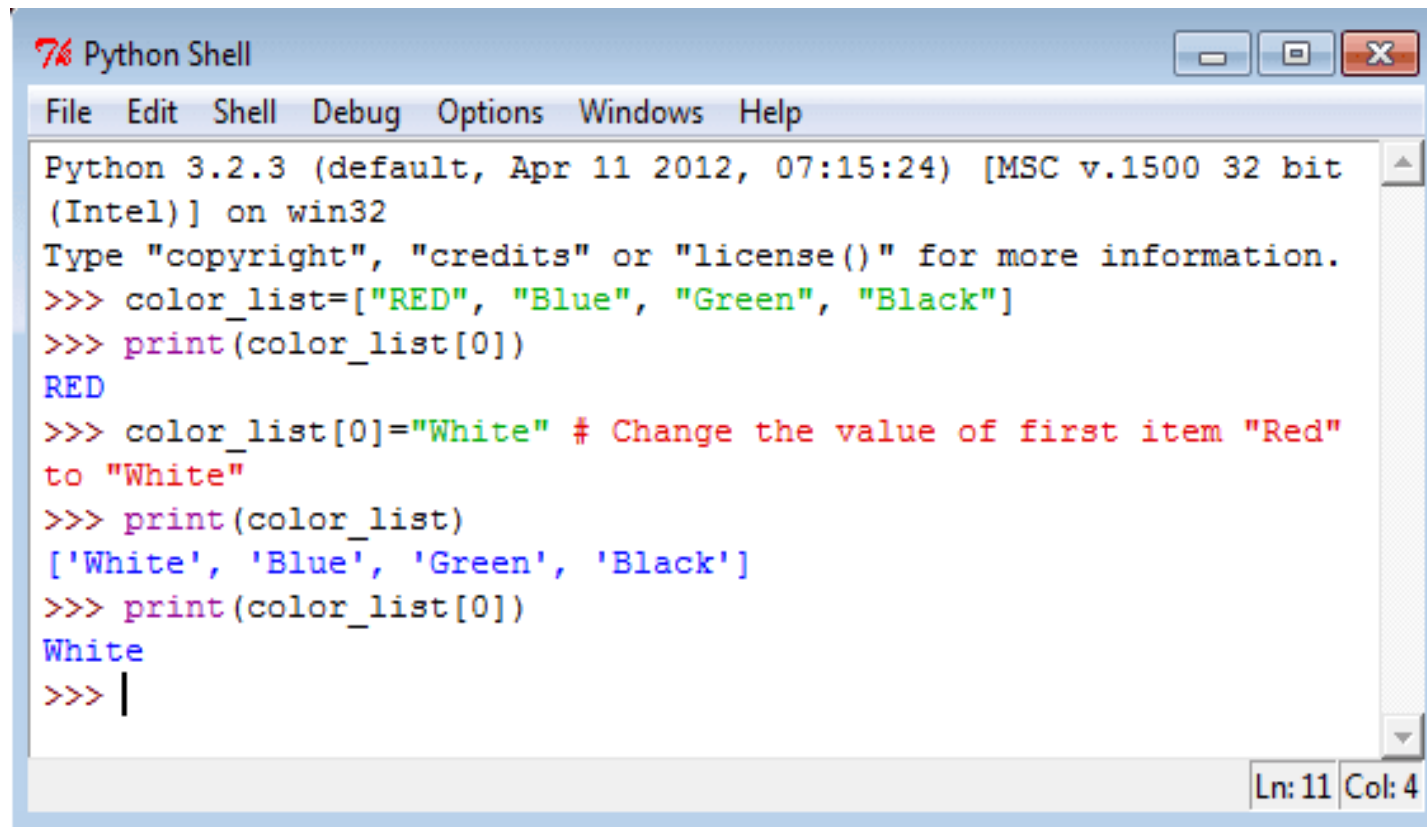


```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> color_list=["RED", "Blue", "Green", "Black"] # The List have
four elements indices start at 0 and end at 3
>>> print(color_list[0:2]) # cut first two items
['RED', 'Blue']
>>> print(color_list[1:2]) # Cut second item
['Blue']
>>> print(color_list[1:-2]) # Cut second item
['Blue']
>>> print(color_list[:3]) # Cut first three items
['RED', 'Blue', 'Green']
>>> print(color_list[:]) # Creates copy of original list
['RED', 'Blue', 'Green', 'Black']
>>> |
```

Ln: 14 Col: 4

List are Mutable

- Items in the list are **mutable** i.e. after creating a list **you can change any item in the list.**

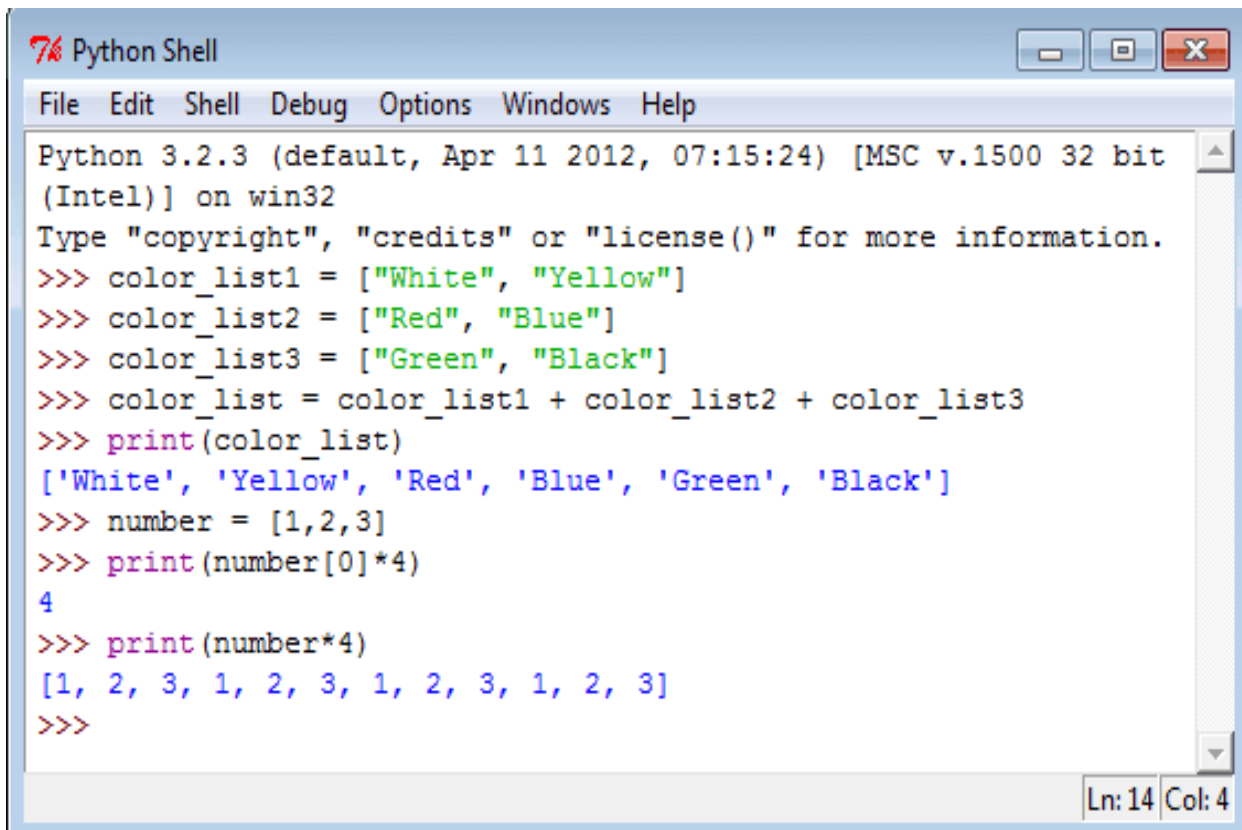
A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> color_list=["RED", "Blue", "Green", "Black"]
>>> print(color_list[0])
RED
>>> color_list[0]="White" # Change the value of first item "Red"
to "White"
>>> print(color_list)
['White', 'Blue', 'Green', 'Black']
>>> print(color_list[0])
White
>>> |
```

The status bar at the bottom right shows "Ln: 11 Col: 4".

Using + and * operators in List

- Use + operator to create a new list that is a **concatenation** of two lists and use * operator to **repeat** a list.

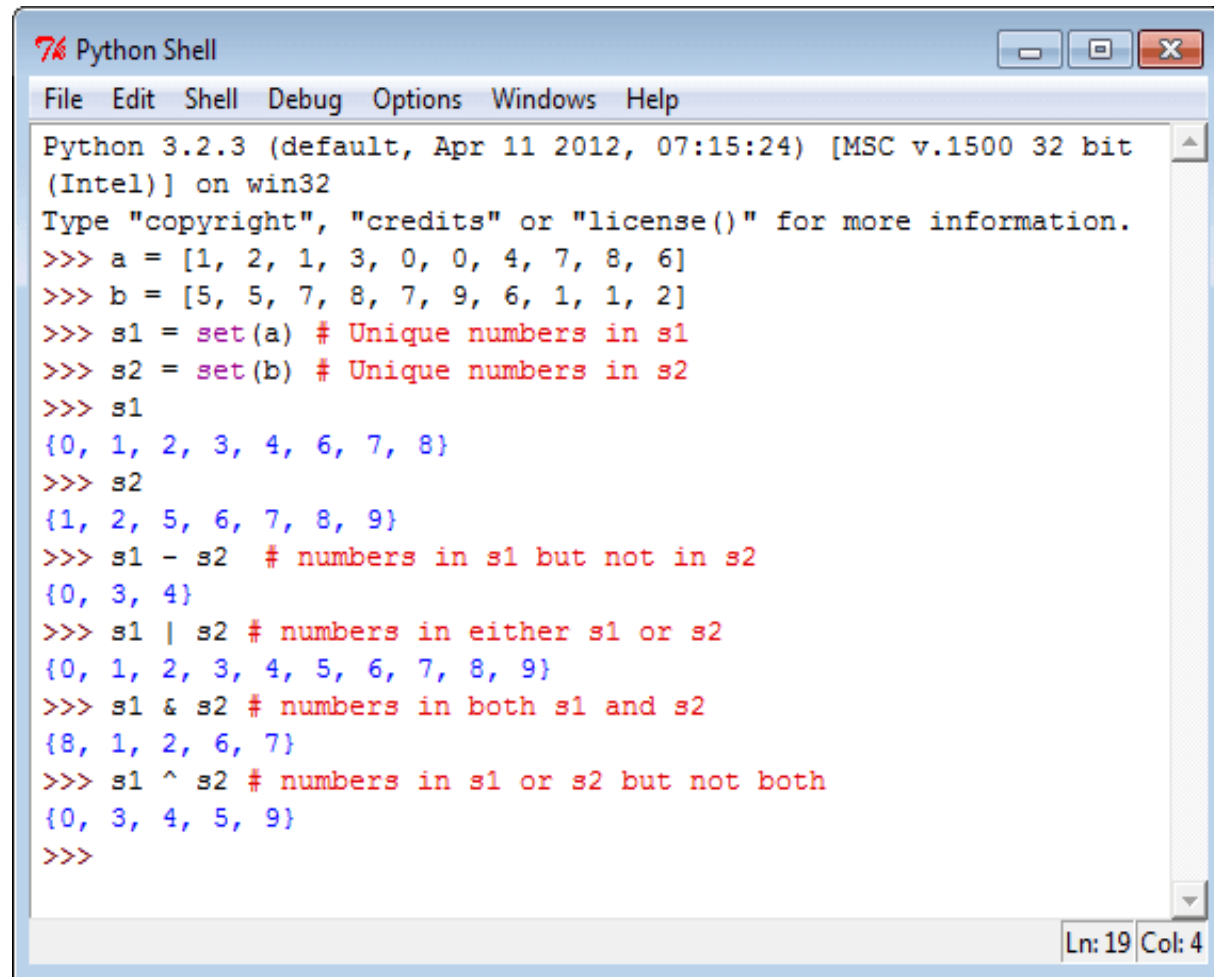
A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> color_list1 = ["White", "Yellow"]
>>> color_list2 = ["Red", "Blue"]
>>> color_list3 = ["Green", "Black"]
>>> color_list = color_list1 + color_list2 + color_list3
>>> print(color_list)
['White', 'Yellow', 'Red', 'Blue', 'Green', 'Black']
>>> number = [1,2,3]
>>> print(number[0]*4)
4
>>> print(number*4)
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
>>>
```

The status bar at the bottom right indicates "Ln: 14 Col: 4".

Set

- A set is an **unordered collection** of unique elements. Basic uses include dealing with set theory (which support mathematical operations like **union**, **intersection**, **difference**, **and** **symmetric difference**) or eliminating duplicate entries.

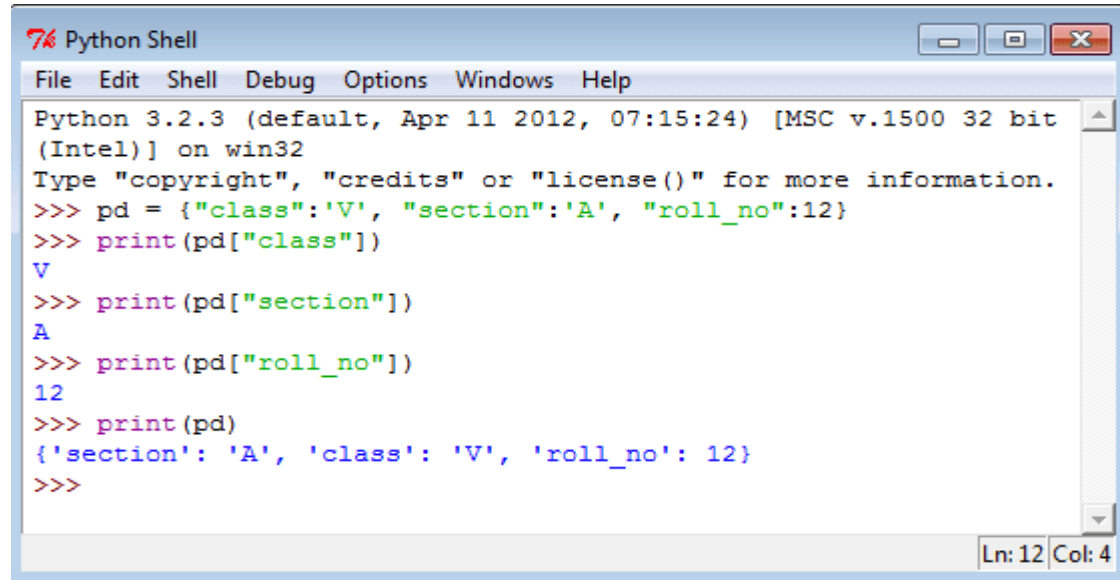
A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> a = [1, 2, 1, 3, 0, 0, 4, 7, 8, 6]
>>> b = [5, 5, 7, 8, 7, 9, 6, 1, 1, 2]
>>> s1 = set(a) # Unique numbers in s1
>>> s2 = set(b) # Unique numbers in s2
>>> s1
{0, 1, 2, 3, 4, 6, 7, 8}
>>> s2
{1, 2, 5, 6, 7, 8, 9}
>>> s1 - s2 # numbers in s1 but not in s2
{0, 3, 4}
>>> s1 | s2 # numbers in either s1 or s2
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
>>> s1 & s2 # numbers in both s1 and s2
{1, 2, 6, 7}
>>> s1 ^ s2 # numbers in s1 or s2 but not both
{0, 3, 4, 5, 9}
>>>
```

The status bar at the bottom right shows "Ln: 19 Col: 4".

Dictionary

- Python dictionary is a **container** of the unordered set of objects like lists.
- The objects are surrounded by curly braces { }.
- The items in a dictionary are a comma-separated list of **key:value pairs** where keys and values are Python data type.
- Each object or value accessed by key and keys are unique in the dictionary. **As keys are used for indexing**, they must be the **immutable** type (string, number, or tuple). You can create an empty dictionary using empty curly braces.

A screenshot of a Python Shell window titled "Python Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Windows", and "Help". The main text area shows the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> pd = {"class": 'V', "section": 'A', "roll_no": 12}
>>> print(pd["class"])
V
>>> print(pd["section"])
A
>>> print(pd["roll_no"])
12
>>> print(pd)
{'section': 'A', 'class': 'V', 'roll_no': 12}
>>>
```

The status bar at the bottom right indicates "Ln: 12 Col: 4".

Python Operators

- **Operators and Operands**
- In computer programming languages **operators** are special symbols which represent **computations, conditional** matching etc. The **values** the operator uses are called **operands**.
- **c = a + b**

Here **a and b are called operands** and **'+' is an operator**

- Python supports following operators.
- ✓ Arithmetic Operators
- ✓ Comparison Operators
- ✓ Logical Operators
- ✓ Assignment Operators
- ✓ Bitwise Operator
- ✓ Membership Operators
- ✓ Operator precedence

Python Arithmetic Operators

Assume variable a holds the value 10 and variable b holds the value 21, then-

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 31$
- Subtraction	Subtracts right hand operand from left hand operand	$a - b = -11$
* Multiplication	Multiplies values on either side of the operator	$a * b = 210$
/ Division	Divides left hand operand by right hand operand	$b / a = 2.1$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 1$
** Exponent	Performs exponential (power) calculation on operators	$a ** b = 10 \text{ to the power } 20$
// Floor Division	-The division of operands where the result is the quotient in which the digits after the decimal point are removed.	$9 // 2 = 4$ and $9.0 // 2.0 = 4.0$

Python Arithmetic Operators

```
a=21
b=10
c=0
c=a+b
print("Line 1- Value of c is",c)
c=a-b
print("Line 2- Value of c is",c)
c=a*b
print("Line 3- Value of c is",c)
c=a/b
print("Line 4- Value of c is",c)
c=a%b
print("Line 5- Value of c is",c)
```

```
a=2
b=3
c=a**b
print("Line 6- Value of c is",c)
a=10
b=5
c=a/b
print("Line 7- Value of c is",c)
```

Output:

```
Line 1- Value of c is 31
Line 2- Value of c is 11
Line 3- Value of c is 210
Line 4- Value of c is 2.1
Line 5- Value of c is 1
Line 6- Value of c is 6
Line 7- Value of c is 2.0
```

Python Comparison Operators

These operators compare the values on either side of them and decide the relation among them. They are also called **Relational operators**. Assume variable a holds the value 10 and variable b holds the value 20, then-

Operator	Description	Example
==	If the values of two operands are equal, then the condition becomes true.	(a == b) is not true.
!=	If values of two operands are not equal, then condition becomes true.	(a != b) is true.
>	If the value of left operand is greater than the value of right operand, then condition becomes true.	(a > b) is not true.
<	If the value of left operand is less than the value of right operand, then condition becomes true.	(a < b) is true.
>=	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.	(a >= b) is not true.
<=	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.	(a <= b) is true.

Python Comparison Operators

```
a = 21
b = 10
if ( a == b ):
    print ("Line 1 - a is equal to b")
else:
    print ("Line 1 - a is not equal to b")

if ( a != b ):
    print ("Line 2 - a is not equal to b")
else:
    print ("Line 2 - a is equal to b")

if ( a < b ):
    print ("Line 3 - a is less than b" )
else:
    print ("Line 3 - a is not less than b")

if ( a > b ):
    print ("Line 4 - a is greater than b")
else:
    print ("Line 4 - a is not greater than b")
```

a,b=b,a #values of a and b swapped. a becomes 10, b becomes 21

```
if ( a <= b ):
    print ("Line 5 - a is either less than or equal to b")
else:
    print ("Line 5 - a is neither less than nor equal to b")

if ( b >= a ):
    print ("Line 6 - b is either greater than or equal to b")
else:
    print ("Line 6 - b is neither greater than nor equal to b")
```

Output:

```
Line 1 - a is not equal to b
Line 2 - a is not equal to b
Line 3 - a is not less than b
Line 4 - a is greater than b
Line 5 - a is either less than or equal to b
Line 6 - b is either greater than or equal to b
```

Python Logical Operators

The following logical operators are supported by Python language. Assume variable a holds True and variable b holds False then-

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is False.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is True.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is True.

Python Assignment Operators

Operator	Description	Example
= Equal to	Assigns values from right side operands to left side operand	<code>c = a + b</code> assigns value of <code>a + b</code> into <code>c</code>
<code>+=</code> Add & Equals to	It adds right operand to the left operand and assign the result to left operand	<code>c += a</code> is equivalent to <code>c = c + a</code>
<code>-=</code> Subtract & Equals to	It subtracts right operand from the left operand and assign the result to left operand	<code>c -= a</code> is equivalent to <code>c = c - a</code>
<code>*=</code> Multiply & Equals to	It multiplies right operand with the left operand and assign the result to left operand	<code>c *= a</code> is equivalent to <code>c = c * a</code>

Python Assignment Operators

Operator	Description	Example
<code>/=</code> Divide & Equals to	It divides left operand with the right operand and assign the result to left operand	<code>c /= a</code> is equivalent to <code>c = c / a</code> <code>c /= a</code> is equivalent to <code>c = c / a</code>
<code>%=</code> Modulus & Equals to	It takes modulus using two operands and assign the result to left operand	<code>c %= a</code> is equivalent to <code>c = c % a</code>
<code>**=</code> Exponent & Equals to	Performs exponential (power) calculation on operators and assign value to the left operand	<code>c **= a</code> is equivalent to <code>c = c ** a</code>
<code>//=</code> Floor Division & Equals to	It performs floor division on operators and assign value to the left operand	<code>c //= a</code> is equivalent to <code>c = c // a</code>

Python Assignment Operators

```
a = 21
```

```
b = 10
```

```
c = 0
```

```
c = a + b
```

```
print ("Line 1 - Value of c is ", c)
```

```
c += a
```

```
print ("Line 2 - Value of c is ", c)
```

```
c *= a
```

```
print ("Line 3 - Value of c is ", c)
```

```
c /= a
```

```
print ("Line 4 - Value of c is ", c)
```

```
c = 2
```

```
c %= a
```

```
print ("Line 5 - Value of c is ", c)
```

```
c **= a
```

```
print ("Line 6 - Value of c is ", c)
```

```
c //= a
```

```
print ("Line 7 - Value of c is ", c)
```

Output:

```
Line 1 - Value of c is 31
```

```
Line 2 - Value of c is 52
```

```
Line 3 - Value of c is 1092
```

```
Line 4 - Value of c is 52.0
```

```
Line 5 - Value of c is 2
```

```
Line 6 - Value of c is 2097152
```

```
Line 7 - Value of c is 99864
```

Python Bitwise Operators

Bitwise operator works on bits and performs bit-by-bit operation. Assume if $a = 60$; and $b = 13$; Now in binary format they will be as follows-

$a = 0011\ 1100$

$b = 0000\ 1101$

$a \& b = 0000\ 1100$

$a | b = 0011\ 1101$

$a \wedge b = 0011\ 0001$

$\sim a = 1100\ 0011$

Python's built-in function `bin()` can be used to obtain binary representation of an integer number.

Python Bitwise Operators

Operator	Description	Example
& Binary AND	Operator copies a bit to the result, if it exists in both operands	(a & b) (means 0000 1100)
Binary OR	It copies a bit, if it exists in either operand.	(a b) = 61 (means 0011 1101)
^ Binary XOR	It copies the bit, if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.
<< Binary Left Shift	The left operand's value is moved left by the number of bits specified by the right operand.	a << = 240 (means 1111 0000)
>> Binary Right Shift	The left operand's value is moved right by the number of bits specified by the right operand.	a >> = 15 (means 0000 1111)

Python Membership Operators

Python's membership operators test for membership in a sequence, such as strings, lists, or tuples. There are two membership operators as explained below-

Operator	Description	Example
in	Evaluates to true, if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true, if it does not find a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.

Python Membership Operators

```
a = 10  
b = 20  
list = [1, 2, 3, 4, 5 ]
```

```
if ( a in list ):  
    print ("Line 1 - a is available in the  
given list") else:  
    print ("Line 1 - a is not available in the  
given list")
```

```
if ( b not in list ):  
    print ("Line 2 - b is not available in the  
given list") else:  
    print ("Line 2 - b is available in the  
given list")
```

```
c=b/a  
if ( c in list ):  
    print ("Line 3 - a is available in the given list")  
else:  
    print ("Line 3 - a is not available in the given list")
```

Output:

```
Line 1 - a is not available in the given list  
Line 2 - b is not available in the given list  
Line 3 - a is available in the given list
```

Operator precedence

- A **data type** is a means of classifying a value and determining what operations can be performed on it. **All objects have a data type.**
- **Operators** are symbols used carry out specific functions/computations.

Operator	Description
**	Exponentiation (raise to the power)
~ + -	Ccomplement, unary plus and minus (method names for the last two are +@ and -@)
* / % //	Multiply, divide, modulo and floor division
+ -	Addition and subtraction
>> <<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise exclusive 'OR' and regular 'OR'
<= < > >=	Comparison operators
<> == !=	Equality operators
= %= /= //= -= +=	Assignment operators
*= **=	
is is not	Identity operators
in not in	Membership operators
not or and	Logical operators

Class	Description
bool	Boolean value
int	integer (arbitrary magnitude)
float	floating-point number
list	mutable sequence of objects
tuple	immutable sequence of objects
str	character string
set	unordered set of distinct objects
frozenset	immutable form of set class
dict	associative mapping (aka dictionary)

String capitalize() Method

It returns a copy of the string with only its first character capitalized.

Syntax: str1.capitalize() **Parameters:** NA

Return Value: string

Example:

```
str1 = "this is string example!!!"  
print ("str1.capitalize() : ", str1.capitalize())
```

Output:

```
str1.capitalize() : This is string example!!!
```

String title() Method

The title() method returns a copy of the string in which first characters of all the words are capitalized.

Syntax: str1.title()

Parameters: NA

Return Value: This method returns a copy of the string in which first characters of all the words are capitalized.

Example:

```
str1 = "this is string example"  
print (str1.title())
```

Output:

This Is String Example

String isalnum() Method

The isalnum() method checks whether the string consists of alphanumeric characters.

Syntax: str1.isalnum()

Parameters: NA

Return Value: This method returns true if all the characters in the string are alphanumeric and there is at least one character, false otherwise.

Example:

```
str1 = "thisis2025" # No space in this string
print (str1.isalnum())
str1 = "this is string example!!!"
print (str1.isalnum())
```

Output:

True

False

String isalpha() Method

The isalpha() method checks whether the string consists of alphabetic characters only or not.

Syntax: str1.isalpha()

Parameters: NA

Return Value: This method returns true if all the characters in the string are alphabetic and there is at least one character, false otherwise.

Example:

```
str1 = "this"; # No space & digit in this string
print (str1.isalpha())
str1 = "this is 2025"
print (str1.isalpha())
```

Output:

True

False

String isdigit() Method

The method isdigit() checks whether the string consists of digits only.

Syntax: str1.isdigit()

Parameters: NA

Return Value: This method returns true if all characters in the string are digits and there is at least one character, false otherwise.

Example:

```
str1 = "123456"; # Only digit in this string  
print (str1.isdigit())  
str1 = "this is string example!!!"  
print (str1.isdigit())
```

Result:

True

False

String islower() Method

The islower() method checks whether all the case-based characters (letters) of the string are lowercase.

Syntax: str1.islower() **Parameters :** NA

Return Value: This method returns true if all cased characters in the string are lowercase and there is at least one cased character, false otherwise.

Example:

```
str1 = "THIS is string example"  
print (str1.islower())  
str1= "this is string example"  
print (str1.islower())
```

Output:

```
False  
True
```

String isupper() Method

The isupper() method checks whether all the case-based characters (letters) of the string are uppercase.

Syntax: str1.isupper()

Parameters: NA

Return Value: This method returns true if all the cased characters in the string are uppercase and there is at least one cased character, false otherwise.

Example:

```
str1 = "THIS IS STRING EXAMPLE"  
print (str1.isupper())  
str1 = "THIS is string example"  
print (str1.isupper())
```

Output:

True

False

String join() Method

The join() method returns a string in which the string elements of sequence have been joined by str separator.

Syntax: str1.join(sequence)

Parameters : sequence - This is a sequence of the elements to be joined.

Return Value: This method returns a string, which is the concatenation of the strings in the sequence seq. The separator between elements is the string providing this method.

Example:

```
s = "-"  
seq = ("a", "b", "c") # This is sequence of strings.  
print (s.join( seq ))
```

Output:

a-b-c

String len() Method

The len() method returns the length of the string.

Syntax: len(str1)

Parameters: string

Return Value: This method returns the length of the string.

Example:

```
str1 = "this is string example!!!"  
print ("Length of the string: ", len(str1))
```

Output: Length of the string: 25

String lower() Method

The method lower() returns a copy of the string in which all case-based characters have been lowercased.

Syntax: str1.lower() **Parameters:** NA

Return Value: This method returns a copy of the string in which all case-based characters have been lowercased.

Example:

```
str1 = "THIS IS STRING EXAMPLE"  
print (str1.lower())
```

Output: this is string example

String upper() Method

The upper() method returns a copy of the string in which all case-based characters have been uppercased.

Syntax : str1.upper() **Parameters:** NA

Return Value: This method returns a copy of the string in which all case-based characters have been uppercased.

Example:

```
str1= "this is string example!!!"  
print ("str1.upper : ",str1.upper())
```

Output:

```
str1.upper : THIS IS STRING EXAMPLE!!!
```

String swapcase() Method

The swapcase() method returns a copy of the string in which all the case-based characters have had their case swapped.

Syntax: str1.swapcase()

Parameters: NA

Return Value: This method returns a copy of the string in which all the case-based characters have had their case swapped.

Example:

```
str1 = "this is string example!!!"  
print (str1.swapcase())  
str1 = "This Is String Example!!!"  
print (str1.swapcase())
```

Output:

```
THIS IS STRING EXAMPLE!!!  
tHIS iS sTRING eXAMPLE!!!
```

String strip() Method

The strip() method returns a copy of the string in which all chars have been stripped from the beginning and the end of the string (default whitespace characters).

Syntax: str1.strip([chars])

Parameters: **chars** - The characters to be removed from beginning or end of the string.

Return Value: This method returns a copy of the string in which all the chars have been stripped from the beginning and the end of the string.

Example:

```
str1 = "*****this is string example!!!*****"  
print (str.strip( '*' ))
```

Output:

```
this is string example!!!
```

String lstrip() Method

The lstrip() method returns a copy of the string in which all chars have been stripped from the beginning of the string (default whitespace characters).

Syntax: str1.lstrip([chars])

Parameters: chars - You can supply what chars have to be trimmed.

Return Value: This method returns a copy of the string in which all chars have been stripped from the beginning of the string (default whitespace characters).

Example: str1 = " this is string example "
print (str1.lstrip())
str1 = "*****this is string example*****"
print (str1.lstrip('*'))

Output:

this is string example
this is string example*****

String rstrip() Method

The rstrip() method returns a copy of the string in which all chars have been stripped from the end of the string (default whitespace characters).

Syntax: str1.rstrip([chars]) **Parameters:** chars - You can supply what chars have to be trimmed.

Return Value: This method returns a copy of the string in which all chars have been stripped from the end of the string (default whitespace characters).

Example:

```
str1 = "    this is string example    "  
print (str1.rstrip())  
str1 = "*****this is string example*****"  
print (str1.rstrip('*'))
```

Output:

```
    this is string example  
*****this is string example
```

String replace() Method

The `replace()` method returns a copy of the string in which the occurrences of old have been replaced with new, optionally restricting the number of replacements to max.

Syntax: `str1.replace(old, new[, max])`

Parameters:

- **old** - This is old substring to be replaced.
- **new** - This is new substring, which would replace old substring.
- **max** - If this optional argument max is given, only the first count occurrences are replaced.

Return Value: This method returns a copy of the string with all occurrences of substring old replaced by new. If the optional argument max is given, only the first count occurrences are replaced.

Example:

```
str1 = "this is string example...!!!this is really string"  
print (str1.replace("is", "was"))  
print (str1.replace("is", "was", 3))
```

Output:

```
thwas was string example...!!! thwas was really string  
thwas was string example...!!! thwas is really string
```

Thank you